HYGIONX: BRIDGING ARTIFICIAL INTELLIGENCE AND EXPLAINABLE MEDICINE

A project report submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING (AI & ML)

by

KANDARI MONISHA	22BFA33110
VAIDYAM LAKSHMI NIHANTH	22BFA33139
KUNCHIPALLLI CHANDHU CHATOPADYAYA	22BFA33118
GIRIRAJU MEHER JAGRUTHI	22BFA33087

Under the guidance of

Dr P. Lokesh Kumar Reddy M.Tech., Ph.D.,

Associate Professor, Dept. of CSE (AI&ML)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)

SRI VENKATESWARA COLLEGE OF ENGINEERING

(AUTONOMOUS)

(Approved by AICTE, New Delhi & Permanently Affiliated to JNTUA, Ananthapuramu
Accredited by NBA, New Delhi & NAAC with 'A' grade)
Karakambadi Road, TIRUPATI – 517507

2022 - 2026

SRI VENKATESWARA COLLEGE OF ENGINEERING

(AUTONOMOUS)

(Approved by AICTE, New Delhi & PermanentlyAffiliated to JNTUA, Ananthapuramu
Accredited by NBA, New Delhi & NAAC with 'A' Grade)
Karakambadi Road, TIRUPATI – 517507.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)

CERTIFICATE

This is to certify that the project work entitled, **"HYGIONX - BRIDGING ARTIFICIAL INTELLIGENCE AND EXPLAINABLE MEDICINE"** is a bonafide record of the project work done and submitted by

KANDARI MONISHA	**22BFA33110**
VAIDYAM LAKSHMI NIHANTH	**22BFA33139**
KUNCHIPALLLI CHANDHU CHATOPADYAYA	**22BFA33118**
GIRIRAJU MEHER JAGRUTHI	**22BFA33087**

under my guidance and supervision for the partial fulfillment of the requirements for the award of B.Tech degree in **COMPUTER SCIENCE AND ENGINEERING (AI & ML).** This project is the result of our own effort and that it has not been submitted to any other University or Institution for the award of any degree or diploma other than specified above

GUIDE **HEAD OF THE DEPARTMENT**

External Viva-Voce Exam held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION

We hereby declare that the project entitled **"HYGIONX - BRIDGING ARTIFICIAL INTELLIGENCE AND EXPLAINABLE MEDICINE"**, done by us under the esteemed guidance of **Dr P.Lokesh Kumar Reddy,** Associate Professor, Dept. of CSE (AI&ML), and is submitted in partial fulfillment of the requirements for the award of the Degree of **Bachelor of Technology in Computer Science and Engineering (AI & ML).**

This report is the result of our own effort and it has not been submitted to any other University or Institution for the award of any degree or diploma other than specified above.

Date:

Place:

KANDARI MONISHA	**22BFA33110**
VAIDYAM LAKSHMI NIHANTH	**22BFA33139**
KUNCHIPALLLI CHANDHU CHATOPADYAYA	**22BFA33118**
GIRIRAJU MEHER JAGRUTHI	**22BFA33087**

ACKNOWLEDGEMENT

We are thankful to out guide **Dr P. Lokesh Kumar Reddy, Associate Professor, Department of CSE (AI & ML),** for his valuable guidance and encouragement. His helping attitude and suggestions have helped in the successful completion of the project report.

We would like to express our greatness and sincere thanks to **Dr R. Swathi, Professor & HOD, Department of CSE (AI&ML),** for her kind help and encouragement during the course of study and in the successful completion of the project work.

We have great pleasure in expressing our hearty thanks to our beloved Principal **Dr Vijaya Gunturu,** for spending his valuable value time with us to complete this project.

Successful completion of any project cannot be done without proper support and encouragement. We sincerely thank the management for providing all the necessary facilities during course of study.

We would like to thank our parents and friends, who have the greatest contributions in all our achievements, for the great care and blessings in making us successful in all our endeavours.

KANDARI MONISHA	**22BFA33110**
VAIDYAM LAKSHMI NIHANTH	**22BFA33139**
KUNCHIPALLLI CHANDHU CHATOPADYAYA	**22BFA33118**
GIRIRAJU MEHER JAGRUTHI	**22BFA33087**

ABSTRACT

HygionX is an AI-powered medical triage assistant designed to provide preliminary health assessment and structured medical guidance through an interactive chatbot interface. The system aims to address the limitations of traditional healthcare access by offering a real-time, accessible, and user-centric solution for symptom evaluation and risk identification. By leveraging Natural Language Processing (NLP) and a hybrid decision-making approach, HygionX interprets user-described symptoms, maintains contextual continuity across interactions, and delivers meaningful healthcare insights. The system incorporates intelligent symptom extraction using both machine learning models and rule-based fallback mechanisms to ensure reliability across different environments. It dynamically engages users through follow-up questioning, enabling progressive refinement of symptom understanding before generating an assessment. Based on the collected information, HygionX performs severity scoring and classifies conditions into risk levels such as Low, Moderate, High, and Emergency, with built-in emergency detection to prioritize critical cases. HygionX emphasizes personalization and continuity by maintaining user-specific health data, including past interactions and medical context, allowing more relevant and adaptive responses over time. The platform supports features such as confidence scoring, symptom tracking, and structured output generation, providing users with clear insights into their health condition along with recommended actions and possible causes. In addition to core triage functionality, the system integrates user authentication, session management and health dashboards. An admin module enables system monitoring, usage analytics, and performance tracking, contributing to overall system efficiency and scalability. HygionX is designed as a scalable web-based application with a modular architecture, ensuring flexibility and ease of integration into modern digital healthcare ecosystems. While the system provides valuable preliminary guidance, it is not intended to replace professional medical consultation but rather to assist users in understanding their symptoms and taking appropriate next steps. Overall, the project presents a comprehensive, intelligent, and user-friendly solution for enhancing accessibility and efficiency in healthcare support systems.

CONTENTS

	Abstract	**i**
	Table of Contents	**ii**
	List of figures	**iv**
	List of Tables	**v**
	Abbreviations	**vi**
Chapter No.	**Description**	**Page No.**
1	**INTRODUCTION**	**1**
2	**PROJECT DESCRIPTION**	**2-4**
	2.1 Problem Definition	2
	2.2 Project Details	3
	2.3 Scope of the Project	4
3	**SYSTEM ANALYSIS**	**5-11**
	3.1 Existing System	5
	3.1.1 Disadvantages	5
	3.2 Proposed System	8
	3.2.1 Advantages	9
4	**ENVIRONMENTS**	**12-16**
	4.1 Software Requirements.	12
	4.2 Hardware Requirements	12
	4.3 Software Features and Dependencies	13
5	**SYSTEM DESIGN**	**17-37**
	5.1 System Architecture	17
	5.2 UML Diagrams	21
	5.2.1 Class Diagram	23
	5.2.2 Sequence Diagram	27
	5.2.3 Activity Diagram	31

Chapter No.	Description	Page No.
	5.2.4 Component Diagram	34
	5.2.5 Deployment Diagram	36
6	**SYSTEM IMPLEMENTATION**	**38-43**
	6.1 Implementation Process	38
	6.2 Module Explanation	41
7	**SYSTEM STUDY AND TESTING**	**44-50**
	7.1 Feasibility Study	44
	7.2 Software Testing	46
8	**SOURCE CODE**	**51-68**
9	**SCREEN LAYOUTS**	**69-72**
	9.1 Interface	69
	9.2 Results	72
10	**FUTURE ENHANCEMENTS**	**73**
11	**CONCLUSION**	**74**
12	**BIBILOGRAPHY**	**75-76**

LIST OF FIGURES

Fig. No.	Figure Name	Page No.
5.1.1	System Architecture	20
5.2.1	Class diagram	27
5.2.2	Sequence diagram	30
5.2.3	Activity diagram	33
5.2.4	Component diagram	35
5.2.5	Deployment diagram	37
6.2.1	List of Modules	41
9.1	Interface	69
9.1.1	Landing Page Interface	69
9.1.2	Authentication Interface	69
9.1.3	Health Assistant Interface	70
9.1.4	Health Timeline Interface	70
9.1.5	Medical Encyclopedia Interface	71
9.1.6	Profile Interface	71
9.2	Results	72
9.2.1	Chat output Interface	72
9.2.2	Confidence scores and reasoning Interface	72

LIST OF TABLES

Table No.	Table Name	Page No.
7.2.1	Unit Testing	47
7.2.2	Integration Testing	47
7.2.3	System Testing	48
7.2.4	Performance Testing	49
7.2.5	Acceptance Testing	50

ABBREVIATIONS

AI	-	Artificial Intelligence
ML	-	Machine Learning
DL	-	Deep Learning
NLP	-	Natural Language Processing
API	-	Application Programming Interface
NER	-	Named Entity Recognition
UML	-	Unified Modelling Language
CBT	-	Cognitive Behavioral Therapy
MySQL	-	My Structured Query Language
REST	-	Representational State Transfer
UI	-	User Interface
DB	-	Database
JSON	-	JavaScript Object Notation
EHR	-	Electronic Health Records
GPU	-	Graphics Processing Unit
TPU	-	Tensor Processing Unit
LLM	-	Large Language Model
HTML	-	Hyper Text Markup Language
CSS	-	Cascading Style Sheets
JS	-	Java Script

Chapter 1
INTRODUCTION

Healthcare is one of the most important sectors in human life. However, in many cases, patients face difficulties in getting timely medical guidance. Traditional healthcare systems often involve long waiting times, lack of immediate consultation, and limited availability of doctors, especially in remote areas. Due to this, people are not able to take quick decisions about their health, even for common symptoms.

Another major issue in healthcare is the lack of personalization. Most systems provide general advice that is not based on the patient's medical history or previous conditions. This leads to less accurate suggestions and sometimes confusion for patients. In addition, many digital healthcare platforms do not clearly explain how they arrive at a particular result. This creates a lack of trust among users.

To solve these problems, our project HygionX introduces an advanced AI-based healthcare chatbot system. This system uses modern technologies such as Natural Language Processing and Machine Learning to understand user queries and provide accurate medical guidance. Unlike traditional systems, HygionX focuses on both personalization and explainability.

The system allows users to enter their symptoms in simple natural language. It then processes the input, identifies important medical terms, and analyzes possible health conditions. Based on this, it provides suggestions, precautions, and next steps. One of the key features of this system is that it explains the reasoning behind its predictions, which helps users understand the results clearly.

HygionX also maintains a secure patient history database. This means the system uses the user information and past history before coming to a conslusion. This ensures continuity of care and improves decision-making.

In addition, the system integrates with external services such as appointment scheduling, medication reminders, and follow-up notifications. This makes the system not just a chatbot, but a complete healthcare assistant.

Overall, HygionX aims to improve the healthcare by making it faster, smarter and more reilable. It plays an important role in enhancing telemedicine and supporting users.

Chapter 2
PROJECT DESCRIPTION

2.1 PROBLEM DEFINITION

In today's digital healthcare environment, many chatbot systems are available to assist users with medical queries. However, these systems have several limitations that reduce their effectiveness and reliability.

One of the major problems is the lack of personalization. Most existing chatbots provide general responses without considering the user's medical history. This leads to inaccurate or incomplete suggestions, which may not be suitable for every individual.

Another important issue is the absence of explainability. Many systems provide answers without explaining how they arrived at the result. This creates confusion and reduces trust among users, as they cannot understand the reasoning behind the suggestions.

Existing systems also struggle with handling complex or rare medical cases. They mainly focus on basic symptom checking and fail to provide accurate results when the situation is complicated. This limits their usefulness in real-world scenarios.

In addition, many chatbots do not perform proper triage analysis. They are unable to determine the severity level of a condition and guide users accordingly. This can lead to incorrect decisions and delays in seeking proper medical help.

Another problem is the lack of continuous patient data tracking. Most systems do not store or utilize past interactions, which means they cannot provide consistent and personalized guidance over time. There are also issues related to limited language support and poor integration with healthcare systems. This reduces accessibility and makes it difficult for users from different backgrounds to use these platforms effectively.

Due to these limitations, there is a need for a more advanced system that can provide personalized, explainable, and reliable healthcare support.

The HygionX project is designed to solve these problems by introducing a system that considers patient history, provides transparent explanations, and delivers accurate medical guidance.

2.2 PROJECT DETAILS

This project, titled "HygionX: AI-Powered Medical Triage Assistant," aims to develop an intelligent healthcare chatbot that automates symptom assessment and provides structured medical guidance using a combination of machine learning and rule-based techniques. The system is designed as a real-time web application that assists users in understanding their health conditions and determining appropriate actions based on symptom severity.

The process begins with a chat-based interface where users describe their symptoms in natural language. The system processes this input using Natural Language Processing (NLP) techniques to extract relevant medical entities. It supports both ML-based extraction and rule-based fallback methods, ensuring consistent performance across different conditions.

HygionX follows a modular architecture consisting of a web interface, backend API layer, triage engine, and database. The backend combines current input with previous symptoms and conversation history to maintain context across interactions, enabling more accurate assessments.

The triage engine evaluates symptoms and determines whether additional information is required. If needed, it generates follow-up questions to gather more details. Once sufficient data is available, the system performs severity scoring and classifies risk levels as Low, Moderate, High, or Emergency. In critical cases, an emergency detection mechanism provides immediate guidance.

The system generates structured outputs including severity level, urgency, recommended actions, and possible conditions. These results are stored along with user sessions, enabling history tracking and continuity. Secure user authentication and profile management support personalized interactions.

By automating the triage process, HygionX reduces manual effort, improves response time, and enhances accessibility to basic healthcare guidance. It provides a scalable and reliable solution that can be integrated into modern digital healthcare systems.

2.3 SCOPE OF THE PROJECT

The project titled "HygionX: AI-Powered Medical Triage Assistant" focuses on developing a digital platform for preliminary health assessment and guidance using AI-driven techniques. The scope of the system primarily covers symptom-based triage, risk evaluation, and structured medical guidance through a real-time chatbot interface.

The system is designed to accept user-reported symptoms in natural language and process them using Natural Language Processing (NLP) techniques. It supports intelligent symptom extraction, context-aware interaction, and multi-turn conversations, enabling continuous tracking of user symptoms within a session. The scope includes dynamic follow-up questioning to gather sufficient information before generating an assessment.

HygionX performs severity scoring and categorizes health conditions into risk levels such as Low, Moderate, High, and Emergency. It also includes an emergency detection mechanism that identifies critical symptoms and prioritizes immediate medical guidance. The system generates structured outputs including severity level, urgency, recommended actions, and possible conditions.

The scope further includes user account management, secure authentication, and storage of medical history and past assessments. Features such as session tracking and health timeline visualization allow users to review previous interactions and maintain continuity in healthcare monitoring. Additionally, the system provides a medical information lookup feature for basic disease awareness and educational support.

However, the system is limited to preliminary assessment and guidance and does not provide definitive medical diagnosis or replace professional healthcare consultation. Its scope is intended to assist users in understanding symptom severity, identifying potential risks, and taking appropriate next steps.

Overall, the project scope emphasizes accessibility, scalability, and real-time interaction, making it suitable for integration into digital healthcare platforms, web applications, and remote health assistance systems.

Chapter 3
SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

Current healthcare chatbots such as Babylon Health, Buoy Health, and Ada Health are primarily designed as symptom-checking systems. These platforms use structured questionnaires and probabilistic reasoning models to map user-reported symptoms to potential conditions. While effective for basic triage, their underlying architecture is largely based on decision trees, rule-based logic, or predefined clinical pathways, limiting adaptability in ambiguous or multi-condition scenarios.

In contrast, mental health-focused chatbots like Wysa, Woebot, and Replika emphasize emotional engagement and conversational continuity. These systems leverage natural language processing (NLP) and cognitive behavioral therapy (CBT)-inspired frameworks. However, they operate within narrow domain boundaries, focusing on emotional well-being rather than integrating physical health diagnostics or clinical data.

Medication and chronic care assistants such as Florence and Molly provide task-oriented support, including reminders, adherence tracking, and basic health monitoring. Similarly, institutional chatbots from organizations like Mayo Clinic and Cleveland Clinic are typically deployed for patient engagement, appointment scheduling, FAQs, and basic symptom guidance within a controlled ecosystem.

3.1.1 DISADVANTAGES

Despite their usefulness,current systems have few technical &functional limitations like:

1. Fragmented Functionality

Most chatbots are single-purpose systems:

- Symptom checkers do not handle emotional or behavioral health.
- Mental health bots lack medical diagnostic capabilities.
- Reminder systems do not perform reasoning or triage.

This creates a disconnected healthcare experience, forcing users to rely on multiple platforms instead of a unified solution.

2. Lack of Longitudinal Patient Context

A major limitation is the absence of integrated Electronic Health Records (EHR):

- Chatbots typically rely only on real-time user input
- No access to past diagnoses, allergies, medications, or lab reports
- No longitudinal tracking of patient health trends

This results in context-blind recommendations, which reduces accuracy and reliability, especially in chronic or complex conditions.

3. Reactive Interaction Model

Most existing chatbots follow a reactive paradigm:

- They respond only when prompted by the user
- No proactive alerts, risk predictions, or early warnings

This limits their role to passive assistants rather than active healthcare companions capable of preventive care.

4. Limited Reasoning Transparency (Black-Box Issue)

AI-driven platforms often lack explainability:

- Users receive outcomes without understanding *why*
- No visibility into symptom weighting, risk scoring, or decision paths

This creates a trust gap, especially in healthcare where decision accountability is critical.

5. Static Decision Frameworks

Many systems rely on:

- Predefined flows
- Limited adaptability to rare or complex cases
- Difficulty handling overlapping symptoms across multiple diseases

This reduces effectiveness in edge cases and multi-morbidity scenarios, which are common in real-world healthcare.

6. Weak Personalization

Although some platforms claim personalization, it is often superficial:

- Based only on age/gender inputs
- No adaptive learning from user behavior over time
- No dynamic risk profiling

True personalization requires continuous learning models, which most current systems lack.

7. Language and Accessibility Barriers

Most chatbots are optimized for English-speaking users:

- Limited multilingual support
- Poor handling of regional dialects or non-standard inputs

This is a serious limitation in countries like India, where linguistic diversity directly impacts accessibility.

8. Poor Integration with Healthcare Ecosystem

Existing chatbots are often standalone systems:

- No integration with hospitals, labs, pharmacies
- No real-time data exchange
- No interoperability with healthcare IT systems

This prevents them from becoming part of a complete digital health infrastructure.

Overall, existing healthcare chatbots demonstrate functional specialization but systemic fragmentation. While they provide value in isolated domains such as symptom checking, mental health support, or medication management, they fail to deliver a holistic, intelligent, and context-aware healthcare experience. Their reliance on static logic, lack of explainability, minimal personalization, and poor integration with clinical systems significantly limits their effectiveness in real-world medical scenarios.

As a result, current systems fall short of building sustained user trust and clinical reliability, highlighting the need for more advanced solutions that combine adaptive intelligence, transparency, and seamless healthcare integration.

3.2 PROPOSED SYSTEM

Our proposed system, HygionX, is an advanced AI-powered healthcare chatbot designed to overcome the limitations of existing systems by integrating context awareness, intelligent reasoning, and continuous personalization into a unified platform. Unlike traditional chatbots that rely solely on real-time inputs, HygionX combines user medical history with live symptom data to generate more accurate, relevant, and personalized healthcare guidance.

HygionX functions as a stateful medical triage system that maintains conversational continuity by tracking user symptoms across multiple interactions. Instead of treating each input independently, the system gathers relevant information through follow-up questions to ensure more reliable assessments.

The system follows a modular architecture consisting of a web interface, backend API layer, triage engine, and database. User inputs are processed by the backend, which reconstructs session context and passes it through a hybrid decision pipeline combining machine learning techniques with rule-based medical logic. This approach ensures both adaptability and safety.The system dynamically adapts its responses based on the amount and quality of information provided by the user, ensuring progressive refinement of results. It also incorporates safety-oriented decision logic to prevent misleading outputs and guide users toward appropriate medical actions.

HygionX performs intelligent symptom extraction using both ML-based methods and rule-based fallback techniques, ensuring consistent performance. Based on extracted symptoms and historical data, the system evaluates severity, determines risk levels, detects emergency conditions, and generates structured medical guidance.

Additionally, the system supports secure user data management, session tracking, and health history storage, enabling continuity across interactions. Overall, HygionX provides a reliable, scalable, and user-centric healthcare assistance solution that improves upon traditional chatbot systems.

3.2.1 ADVANTAGES

1. Intelligent Symptom Understanding and NLP Layer

HygionX leverages state-of-the-art Natural Language Processing models, including MiniLM (via HuggingFace Transformers) for semantic sentence encoding and SciSpacy NER (Named Entity Recognition) for extracting medical entities such as symptoms, conditions, and medications.

- MiniLM enables context-aware interpretation of user inputs, even when phrased informally
- SciSpacy identifies and standardizes clinical terms from unstructured text
- Supports flexible, conversational input instead of rigid questionnaires

This allows the system to move beyond keyword matching and achieve deep linguistic understanding of patient concerns.

2. Hybrid Decision Engine (AI + Rule-Based Safety Layer)

Instead of relying purely on AI predictions, HygionX uses a hybrid architecture:

- Pre-trained machine learning models for disease prediction
- Predefined medical safety rules to validate and constrain outputs

This ensures:

- Higher accuracy and consistency
- Prevention of unsafe or misleading recommendations
- Alignment with basic clinical guidelines

The system outputs structured responses in JSON format, enabling easy integration with frontend systems and future scalability.

3. Context-Aware Personalization

A key innovation in HygionX is its ability to incorporate longitudinal patient data:

- User medical history (past symptoms, diagnoses)
- Medication records and allergies
- Previous chatbot interactions

All data is securely stored in a MySQL database, enabling:

- Continuity across sessions
- Adaptive learning based on user patterns
- More precise and personalized recommendations over time

This transforms the chatbot from a stateless responder into a persistent health assistant.

4. Emergency Detection & Safety mechanism

To ensure user safety, HygionX includes a built-in emergency system:

- Identifies critical symptoms such as chest pain, breathing difficulty, or loss of consciousnes
- Immediately classifies such cases as Emergency
- Overrides normal processing to provide urgent guidance
- Triggers highest severity scoring and prioritizes immediate medical attention recommendations

This feature ensures timely intervention and prioritizes user safety in high-risk scenarios.

5. Proactive and Preventive Healthcare Features

Unlike traditional reactive chatbots, HygionX introduces proactive intelligence:

- Tracks symptom patterns over time
- Identifies potential health risks
- Encourages early intervention

Additionally, the system includes a safety-trigger mechanism:

- If prediction confidence falls below a threshold, it auto recommends doctor.

This ensures the system remains assistive, not authoritative, reducing medical risk.

6. Severity Scoring and Risk Classification

The system evaluates the seriousness of a condition using a structured scoring mechanism:

- Assigns severity score based on detected symptoms
- Maps scores to risk levels (Low/Moderate/Urgent/Emergency)

- Adjusts results dynamically based on symptom intensity and combinations
- Incorporates emergency overrides to ensure critical cases receive maximum severity

This allows users to clearly understand urgency of their condition and take appropriate action.

HygionX represents a significant shift from traditional reactive and single-purpose healthcare chatbots toward a holistic, intelligent, and context-aware digital healthcare assistant. Unlike conventional systems that operate on isolated inputs and predefined flows, HygionX integrates advanced Natural Language Processing (NLP, and secure data management to deliver highly personalized and reliable healthcare guidance.

HygionX leverages transformer-based NLP and medical entity recognition to interpret complex, unstructured user inputs and convert them into meaningful clinical insights. By integrating user medical history, past interactions, and real-time symptoms, it delivers dynamic, personalized recommendations instead of static responses. The inclusion of Explainable AI ensures transparency by providing reasoning, key factors, and confidence levels behind each prediction, improving user trust and decision-making.

The system also ensures safety through a hybrid approach that combines machine learning with predefined medical rules and confidence-based triggers, recommending doctor consultation when needed. With secure data storage and structured JSON outputs, HygionX supports continuous monitoring, scalability, and seamless integration, making it a reliable and adaptive healthcare assistant.

Chapter 4
ENVIRONMENTS

4.1 SOFTWARE REQUIREMENTS

Operating System	: Windows 10/11
Browser	: Google Chrome/ Firefox
Programming language	: Python
Frontend	: HTML, CSS, JS
Backend Framework	: Flask (REST API)
Databases	: MySQL - user records
Authentication	: Firebase
NLP/ML Libraries	: PyTorch, HuggingFace Transformers
Development Tools	: VS Code, Google Colab, Jupyter Notebook
Version Control	: Git, GitHub

4.2 HARDWARE REQUIREMENTS

Processor	: Intel Core 2 Duo / i3 Processor (1.8 GHz or higher)
RAM	: 4GB minimum (8GB recommended)
Hard disk	: 20 GB or more
Display	: Standard HD Monitor
Keyboard	: Standard 104-keys keyboard
Mouse	: Optical mouse
Internet	: Stable broadband connection

4.3 SOFTWARE FEATURES AND DEPENDENCIES

1. Python (Core Programming Language)

Python serves as the backbone of the HygionX system due to its extensive support for AI, data processing, and web development. It enables seamless integration of machine learning models, APIs, and databases within a single environment. Its large ecosystem of libraries significantly reduces development time while maintaining scalability and performance.

2. Flask (Backend Framework)

Flask is a lightweight and flexible web framework used to build the RESTful API layer of HygionX. It handles:

- Incoming user requests from the frontend
- Communication with NLP models and decision engine
- Data exchange between server and database

Flask's modular structure allows easy integration of AI components and supports rapid development of scalable backend services.

3. HTML, CSS, JavaScript (Frontend Technologies)

These technologies are used to create an interactive and user-friendly chatbot interface:

- HTML structures the chatbot layout
- CSS enhances visual presentation and responsiveness
- JavaScript enables real-time communication with the Flask API using asynchronous requests (AJAX/fetch)

Together, they ensure a smooth and engaging user experience.

4. MySQL (Database System)

MySQL is used as a relational database to store and manage:

- User profiles and authentication-linked data
- Medical history and past interactions
- Session logs and chatbot responses

It ensures data consistency, efficient querying, and long-term storage, which is essential for personalization and continuity.

5. Firebase Authentication

Firebase provides a secure and reliable authentication mechanism:

- User registration and login
- Session management
- Token-based authentication

It enhances system security by ensuring that only authorized users can access and modify their health data.

6. PyTorch (Deep Learning Framework)

PyTorch is used for implementing and running machine learning models within the system. It provides:

- Efficient tensor computations
- Model loading and inference capabilities
- Flexibility for future model training and fine-tuning

Its dynamic computation graph makes it suitable for research-oriented and adaptive AI systems like HygionX.

7. HuggingFace Transformers

This library provides access to pre-trained transformer models such as MiniLM, which are used for:

- Semantic sentence embeddings
- Understanding user intent
- Improving contextual interpretation of inputs

It significantly enhances the chatbot's ability to process natural, conversational language.

8. SciSpacy (Medical NLP - NER)

SciSpacy is a specialized NLP library for biomedical text processing. It is used for:

- Named Entity Recognition (NER)
- Extracting symptoms, diseases, and medical terms from user input

This ensures that raw user text is converted into structured medical data, improving prediction accuracy.

9. Google Colab

Google Colab is used for:

- Training and testing machine learning models
- Running experiments on datasets
- Utilizing cloud-based GPU/TPU resources

It accelerates development by providing a high-performance computing environment without local hardware limitations.

10. Jupyter Notebook

Jupyter Notebook supports:

- Data preprocessing and visualization
- Model experimentation and debugging
- Step-by-step execution of code

It is particularly useful during the research and prototyping phase of the project.

11. Visual Studio Code (VS Code)

VS Code is the primary development environment used for:

- Writing and managing code
- Debugging backend and frontend components
- Integrating extensions for Python, Git, and web development

Its lightweight nature and rich plugin ecosystem improve developer productivity.

12. Git (Version Control System)

Git is used to:

- Track changes in the codebase

- Manage different versions of the project
- Enable collaboration and rollback if needed

It ensures code integrity and organized development workflow.

13. GitHub (Repository Hosting Platform)

GitHub is used to:

- Store and manage project code remotely
- Collaborate with team members
- Maintain version history and documentation

It also supports project visibility and deployment readiness.

The combination of these tools enables HygionX to function as a robust, scalable, and intelligent healthcare system, where each technology plays a clearly defined role in strengthening overall system performance. The integration of Python with Flask ensures efficient backend processing and seamless communication between components, while advanced NLP and machine learning libraries such as PyTorch and HuggingFace Transformers enhance the system's ability to understand, analyze, and respond to complex user inputs with high accuracy.

At the same time, the use of MySQL and Firebase Authentication ensures secure data storage, user privacy, and controlled access, which are critical in handling sensitive healthcare information. Frontend technologies like HTML, CSS, and JavaScript contribute to an intuitive and responsive user interface, improving accessibility and user engagement. Development tools such as VS Code, Jupyter Notebook, and Google Colab streamline the processes of coding, testing, and model optimization, while Git and GitHub support efficient version control and collaborative development.

Together, this well-integrated technology stack enables HygionX to deliver real-time processing, continuous learning, secure data management, and scalable deployment, making it capable of handling growing user demands while maintaining reliability, accuracy, and a high standard of user experience in AI-driven healthcare support.

Chapter 5

SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE

The HygionX system is designed using a modular, scalable, and secure client-server architecture that ensures efficient communication between different components while maintaining high performance and data integrity. The architecture is divided into multiple layers, each responsible for specific functionalities such as user interaction, data processing, intelligence computation, and storage. This layered design improves maintainability, scalability, and security of the system.

The overall architecture follows a three-tier model consisting of the Presentation Layer, Application Layer, and Data Layer. The Presentation Layer acts as the user interface, allowing users to interact with the system through a web-based chat interface. It is developed using HTML, CSS, and JavaScript, providing a user-friendly and responsive design. Users can enter symptoms in natural language and receive responses in a conversational format. The interface also displays explanations, recommendations, and confidence levels, ensuring clarity and ease of use.

The system integrates a secure authentication mechanism using Firebase Authentication, which ensures that only authorized users can access the platform. This layer handles user registration, login, and session management while protecting sensitive medical data. It also supports role-based access control, allowing differentiation between normal users and administrators. By implementing secure authentication, the system maintains data privacy and prevents unauthorized access.

The Application Layer serves as the core operational component of the system and is implemented using the Flask framework. It handles API requests, manages sessions, and coordinates communication between different modules. When a user submits a query, the backend processes the request and forwards it to the Healthcare Intelligence Engine. After processing, the generated response is sent back to the user interface. This layer ensures smooth data flow and efficient system operation through RESTful APIs.

At the heart of the system lies the Healthcare Intelligence Engine, which is responsible for analyzing user input and generating meaningful medical insights. This module uses Natural Language Processing techniques powered by HuggingFace Transformers such as MiniLM to understand user queries. It also employs Named Entity Recognition using SciSpacy to extract relevant medical entities such as symptoms and conditions. The system then performs triage and risk classification, categorizing cases into low, moderate, or high risk. Based on predefined rules and trained models, it generates recommendations and provides explanations along with confidence scores, ensuring transparency and trust.

The Knowledge Retrieval Module enhances the system by providing additional medical information to users. It fetches relevant data from external APIs or medical datasets and presents it in a simplified format. This helps users gain a better understanding of their conditions and supports informed decision-making. By combining prediction with knowledge support, the system becomes more informative and user-centric.

The Conversation and Session Management Module plays a crucial role in maintaining context throughout user interactions. It tracks chat history, maintains session continuity, and ensures that follow-up questions are relevant. By linking multiple messages and storing previously detected symptoms, this module improves the accuracy and consistency of responses, enabling a personalized user experience.

The Data Layer is responsible for storing and managing all system data using a structured MySQL database. It stores user profiles, medical history, chat logs, detected symptoms, and triage results. This structured storage enables efficient data retrieval and ensures consistency and reliability. By maintaining historical data, the system can provide personalized recommendations and track user health over time.

In addition to internal components, the system integrates with external services such as appointment scheduling APIs, medication reminder systems, and notification services. These integrations extend the functionality of the system and make it more practical for real-world healthcare applications. They also support features like follow-up notifications and telemedicine services.

The data flow in the system follows a structured process. The user inputs symptoms through the chat interface, and the request is sent to the backend. After authentication verification, the input is processed by the NLP module, where symptoms are extracted and analyzed. The Healthcare Intelligence Engine then evaluates the data, generates predictions, and assigns a risk level. The results are stored in the database and sent back to the user as a structured response. This ensures real-time processing and accurate output.

The architecture is designed with scalability and performance in mind. Its modular structure allows independent components to be updated or expanded without affecting the entire system. The use of lightweight frameworks, efficient machine learning models, and optimized database queries ensures fast response times even with increasing user load. The system is also cloud-ready, making it suitable for future deployment and scaling.

To ensure reliability, the system incorporates fault tolerance mechanisms such as error handling, backup storage, and logging. These features allow the system to handle failures gracefully without disrupting user experience. Additionally, strong security measures, including encrypted data storage, secure API communication, and controlled access, protect sensitive healthcare data and maintain user trust.

In conclusion, the HygionX system architecture is a robust and intelligent framework that combines modern web technologies with advanced artificial intelligence techniques. Its modular design, secure infrastructure, and explainable AI capabilities enable it to deliver personalized and reliable healthcare support. The architecture is flexible and future-ready, allowing integration of advanced features such as voice interaction, wearable device connectivity, and real-time telemedicine services.

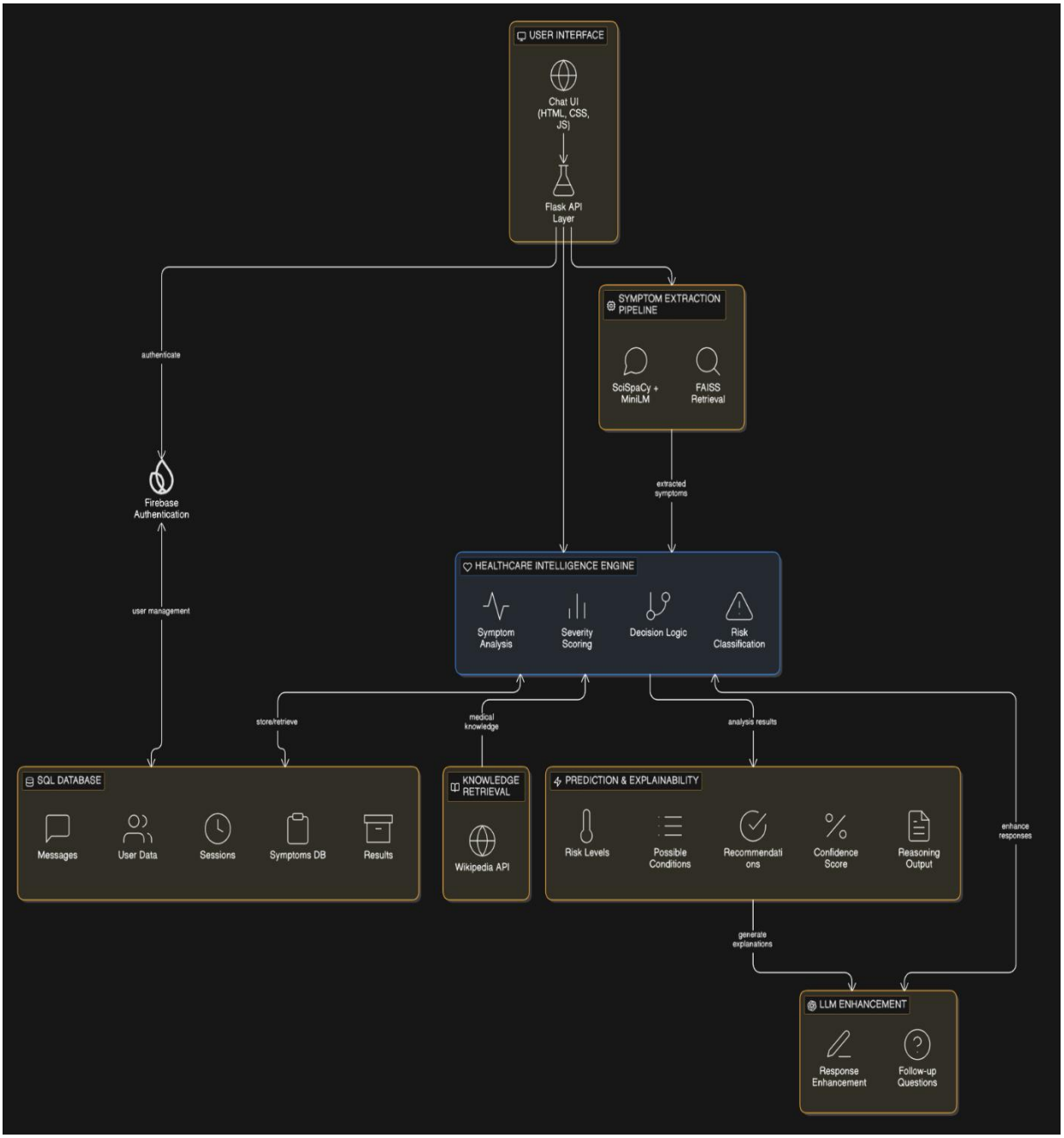

Fig 5.1.1 : System Architecture

The HygionX architecture ensures seamless integration of user interaction, AI-driven analysis, and secure data management. It enhances healthcare accessibility by providing personalized, explainable, and real-time medical guidance. The system's scalable and modular design makes it adaptable for future advancements in digital healthcare.

5.2 UML DIAGRAMS

What is UML?

Unified Modeling Language (UML) is a standardized visual modeling language used to represent the structure, behavior, and interactions of a system. It helps in designing, analyzing, and documenting software systems through various types of diagrams. UML provides a clear blueprint of how system components interact, making it easier for developers, designers, and stakeholders to understand complex workflows.

In the context of the HygionX system, UML is used to represent how different modules such as the User Interface, Flask API, Firebase Authentication, Healthcare Intelligence Engine, Knowledge Retrieval, and Database interact with each other. It visually explains the flow of data from user input to AI processing and response generation, including symptom extraction, risk classification, and explainability outputs. By converting the system architecture into UML diagrams, the entire workflow becomes easier to understand and implement.

Contents of UML:

UML consists of different types of diagrams that represent various aspects of a system. These include:

- **Use Case Diagram:**

Represents the interaction between users (actors) and the system. In HygionX, it shows how users input symptoms, receive responses, and manage profiles.

- **Class Diagram:**

Describes the structure of the system by showing classes such as User, Session, Symptoms, and Results along with their relationships.

- **Sequence Diagram:**

Illustrates the step-by-step flow of messages between components, such as how a user query moves from the interface to the backend and AI engine.

- **Activity Diagram:**

Represents the workflow of processes like symptom analysis, risk classification, and response generation.

- **Component Diagram:**

Shows how different system modules (frontend, backend, database, AI engine) are organized and interact.

- **Deployment Diagram:**

Describes how the system is deployed across hardware and software environments, including servers, databases, and cloud services.

These diagrams collectively help in understanding both the static and dynamic aspects of the HygionX system.

Advantages of UML

UML provides several benefits in system design and development:

- Improves Visualization: Helps in clearly representing complex system architecture in an easy-to-understand graphical format.
- Better Communication: Acts as a common language between developers, designers, and stakeholders.
- Simplifies Complexity: Breaks down complex processes like NLP processing and decision-making into understandable diagrams.
- Enhances System Design: Helps in planning and organizing system components efficiently.
- Supports Documentation: Provides proper documentation for future reference and maintenance.
- Facilitates Debugging and Development: Makes it easier to identify errors and improve system workflow.
- Scalability and Flexibility: Helps in modifying and extending the system architecture as new features are added.

In this project five basic UML diagrams have been explained:

1. Class Diagram
2. Sequence Diagram
3. Activity Diagram
4. Component Diagram
5. Deployment Diagram

5.2.1 Class Diagram

A Class Diagram is one of the most important UML diagrams used to represent the **static structure of a system**. It provides a detailed view of how a system is organized by illustrating its classes, attributes, methods, and the relationships between them. Unlike dynamic diagrams that focus on system behavior over time, class diagrams act as a **blueprint of the system**, helping developers understand the design before implementation.

A class diagram simplifies complex systems by visually organizing components and showing how they interact at a conceptual level. It plays a crucial role in software development, database design, and system planning. In the context of the HygionX system, the class diagram represents core entities such as **User, Session, Message, TriageResult, and the Healthcare Intelligence Engine (HygionXMediAITriage),** which together manage user interaction, data storage, and AI-based healthcare analysis.

1. Classes

Classes are the fundamental building blocks of a class diagram, representing real-world entities or system components.

In HygionX:

- User represents patient information
- Session represents each interaction between user and system
- Message represents chat communication

- TriageResult stores AI-generated outputs
- HygionXMediAITriage represents the AI processing engine

These classes define the overall structure of the healthcare chatbot system.

2. Attributes

Attributes represent the **data stored within each class**.

Examples:

- User → id, email, name, known_conditions
- Session → id, user_id, title
- Message → id, session_id, role, content
- TriageResult → severity_score, risk_level, recommended_action

These attributes help store user profiles, conversation history, and medical analysis results, enabling personalized healthcare support.

3. Methods (Operations)

Methods define the **behavior or functionality** of a class.

In HygionX:

- extract_symptoms() → identifies symptoms using NLP
- calculate_severity_score() → evaluates severity
- generate_final_assessment() → produces final output
- process_message() → handles user queries

These methods are mainly part of the Healthcare Intelligence Engine and represent the system's core AI processing capabilities.

4. Relationships

Relationships define how classes are connected and interact with each other.

- Association: A User can have multiple Sessions, and each Session contains multiple Messages.
- Multiplicity: One User → Many Sessions (1 to *)
  One Session → One TriageResult (1 to 1)
- Dependency: The AI Engine (HygionXMediAITriage) depends on session data to generate results.

These relationships reflect real-world interactions and ensure logical system design.

5. Encapsulation

Encapsulation combines data and methods within a single class.

Example:
HygionXMediAITriage includes both:

- Data → session_symptoms, follow_up_count
- Methods → symptom extraction and analysis

This improves modularity and protects data integrity.

6. Abstraction

Abstraction hides complex internal logic and shows only essential details.

In HygionX:

- Users see only outputs like risk level and recommendations
- Internal NLP and ML processing remains hidden

This simplifies user interaction while maintaining powerful backend functionality.

7. Modularity and Reusability

The system is divided into independent classes, making it easier to:

- Maintain and update
- Extend with new features
- Reuse components in future development

This ensures scalability and flexibility of the system.

8. Real-world Mapping

The class diagram closely represents real-world healthcare processes:

- User → Patient
- Session → Consultation
- Message → Conversation
- TriageResult → Diagnosis/Advice
- AI Engine → Medical decision system

This makes the system intuitive and logically structured.

The class diagram of HygionX provides a comprehensive representation of the system's structure by clearly defining all core components involved in the healthcare chatbot. It identifies key classes such as User, Session, Message, TriageResult, and the Healthcare Intelligence Engine, along with their roles in the system. Each class includes attributes that store important data, such as user details, chat history, and medical analysis results. It also defines methods that represent system functionalities like symptom extraction, severity calculation, and response generation.

The relationships between these classes, such as associations and multiplicity, help in understanding how different components interact with each other. The diagram also supports object-oriented principles like encapsulation and abstraction, making the system modular and easier to manage.

By acting as a blueprint, the class diagram guides developers during implementation and reduces design complexity. It helps in identifying errors early and ensures consistency in development. Overall, it plays a crucial role in building a scalable, efficient, and well structured healthcare system.

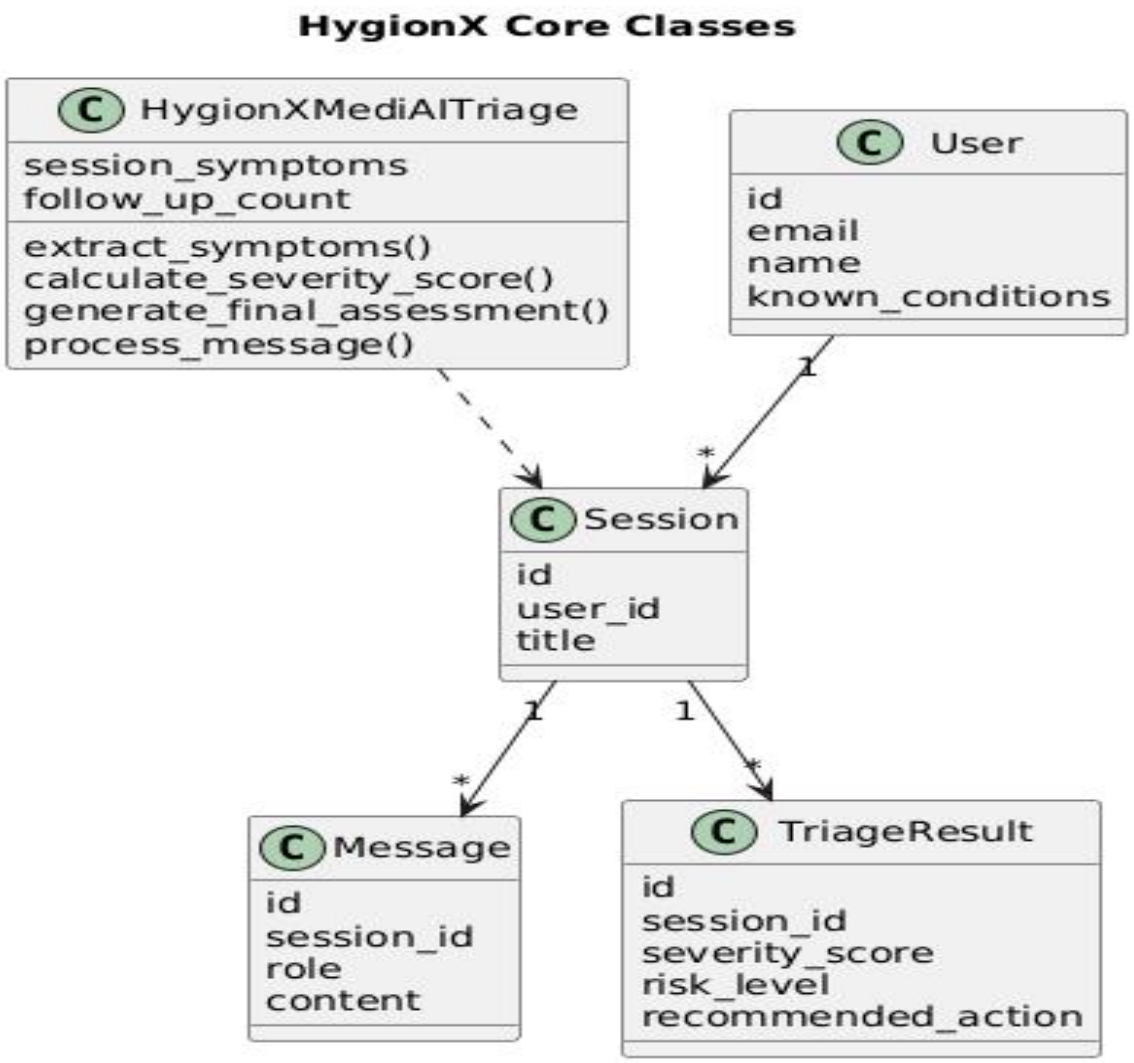

Fig 5.2.1: Class diagram

5.2.2 Sequence Diagram

A Sequence Diagram is a type of UML diagram used to represent the dynamic behavior of a system by showing how different components interact with each other over time. It focuses on the order of messages exchanged between system elements and clearly illustrates the flow of execution from start to end.

This makes the sequence diagram a very useful tool for understanding how a system works internally, especially for processes that involve multiple components communicating with each other. It helps in visualizing real-time interactions, identifying system workflows, and improving system design.

In the context of the HygionX system, the sequence diagram represents how a user interacts with the chatbot and how the request flows through various components such as the User Interface (UI), API (Flask backend), Symptom Extraction Pipeline, Healthcare Intelligence Engine (Triage), and Database (DB).

Dynamic Behaviour in HygionX (Using Diagram)

The sequence diagram shows the step-by-step interaction process:

1. The User enters symptoms through the UI.
2. The UI sends a request to the backend using a POST API call (/api/chat).
3. The API loads the user session and chat history from the database.
4. The request is forwarded to the Symptom Extraction Pipeline, which processes the input using NLP techniques.
5. The processed data is passed to the Triage Engine, where functions like process_message() are executed.

At this point, the system follows two possible paths:

a. Follow-up Case

- If more information is needed, the system generates a follow-up question
- The message and state are saved in the database
- A follow-up JSON response is sent back to the user

b. Final Assessment Case

- If sufficient data is available, the system generates:
- Severity score
- Urgency level
- Medical advice
- The results are stored in the database
- A final triage JSON response is returned

Finally, the UI displays the response to the user.

This clearly shows how the system dynamically processes user input and generates intelligent healthcare responses.

Key Elements of Sequence Diagram

- Actors: User interacting with the system
- Lifelines: UI, API, Pipeline, Triage, Database
- Messages: API calls, processing requests, responses
- Activation Bars: Show active processing
- Conditions (alt block): Represent decision-making (follow-up vs final result)

Steps to Create a Sequence Diagram

1. **Identify Actors and Components**
   Determine who interacts with the system (e.g., User) and system components (UI, API, AI Engine, Database).
2. **Define the Scenario**
   Choose a specific process to represent, such as "User entering symptoms and receiving diagnosis."
3. **List Interactions in Order**
   Write down the sequence of steps from input to output (API calls, processing, response).
4. **Draw Lifelines**
   Represent each component (UI, API, Pipeline, etc.) as vertical lines.
5. **Add Messages/Arrows**
   Show communication between components using arrows in time order.
6. **Include Conditions (alt/loop blocks)**
   Represent decision points like follow-up questions or final assessment.
7. **Add Return Responses**
   Show how results are sent back to the user.
8. **Refine and Simplify**
   Ensure the diagram is clear, readable, and logically correct

The sequence diagram of HygionX clearly illustrates the dynamic interaction between system components during a user request. It shows how data flows step-by-step from the user interface to the backend, AI modules, and database. The diagram highlights key decision points such as follow-up questions or final assessment generation. It helps in understanding real-time communication between components like API, pipeline, and triage engine.

Additionally, it simplifies complex system behavior and improves communication among developers and stakeholders. It is useful for debugging, testing, and optimizing system performance. It also helps identify bottlenecks and ensures smooth workflow execution. Overall, the sequence diagram provides a clear, structured, and efficient view of system operations.

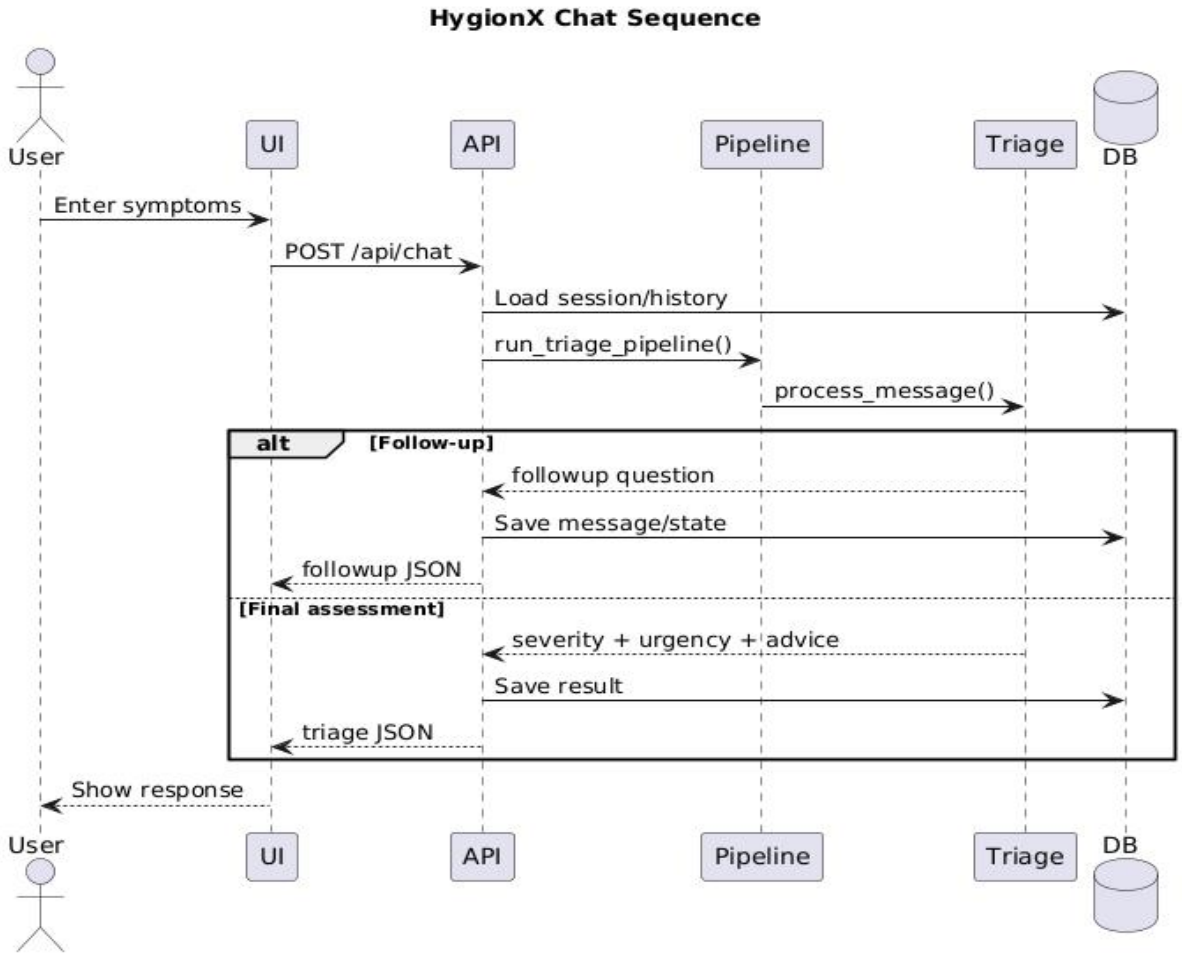

Fig 5.2.2: Sequence diagram

5.2.3 Activity Diagram

Activity Diagram – HygionX Triage Flow The Activity Diagram represents the workflow and step-by-step execution process of the HygionX system when a user interacts with the healthcare chatbot. It is used to model the dynamic behavior and control flow of the system, showing how activities are performed sequentially and how decisions are made at different stages.

The process begins when the user sends a symptom message through the interface. The system first performs input validation to ensure that the provided data is accurate and meaningful. After validation, the system loads the session context, which includes previous chat history and stored user information. This helps maintain continuity and enables personalized responses.

Next, the system executes the triage pipeline, where the input is processed using NLP techniques. During this stage, symptoms are extracted and merged from both the current message and past interactions. This ensures that the system has a complete understanding of the user's condition before proceeding further.

A key decision point in the workflow is determining whether a follow-up question is needed. If sufficient information is not available, the system generates a follow-up question and returns it to the user. This step may repeat until enough data is collected, forming a loop that improves the accuracy of the final result.

If no follow-up is required, the system proceeds to severity scoring, where it evaluates the seriousness of the condition. Based on this evaluation, the system assigns an urgency level (low, moderate, or high) and builds a complete assessment that includes possible conditions and recommended actions.

Another important decision point is checking whether the situation is an emergency. If classified as an emergency, the system generates an immediate response advising the user to seek urgent medical attention. Otherwise, it provides a standard triage response with appropriate medical guidance.

Finally, all session data and results are stored in the database, including symptoms, analysis, and responses. The process ends after saving the information, ensuring it can be used for future interactions and improved personalization.

Key Features of the Activity Diagram:

- Represents the complete workflow from user input to final output
- Shows important decision points such as follow-up handling and emergency detection
- Demonstrates conditional and iterative flows within the system
- Provides a clear understanding of system logic and execution steps
- Helps identify dependencies and process control mechanisms

Elements Identified Before Drawing :

- Activities (Actions): Steps like validation, symptom extraction, and scoring
- Actors/Participants: User and system modules involved in the process
- Start and End Points: Defines where the workflow begins and ends
- Control Flow: Sequence of operations in the system
- Decision Points: Conditions like follow-up required or emergency detection
- Inputs and Outputs: Data processed and generated at each stage
- Loops/Iterations: Repeated steps such as follow-up questioning

In conclusion, the activity diagram of HygionX provides a clear and structured visualization of the system's operational flow, from receiving user input to generating and storing results. It effectively represents how different processes are connected and executed in a logical sequence. The diagram highlights key decision-making points, such as follow-up handling and emergency detection, ensuring that the system responds appropriately in different scenarios.

It also helps in understanding how data flows through various stages, including validation, symptom extraction, analysis, and response generation. By clearly showing each step, it simplifies complex system behavior and improves overall clarity. Additionally, it supports better system design, debugging, and performance optimization while enhancing communication among developers and stakeholders. Overall, it contributes to building a reliable, efficient, and scalable healthcare system.

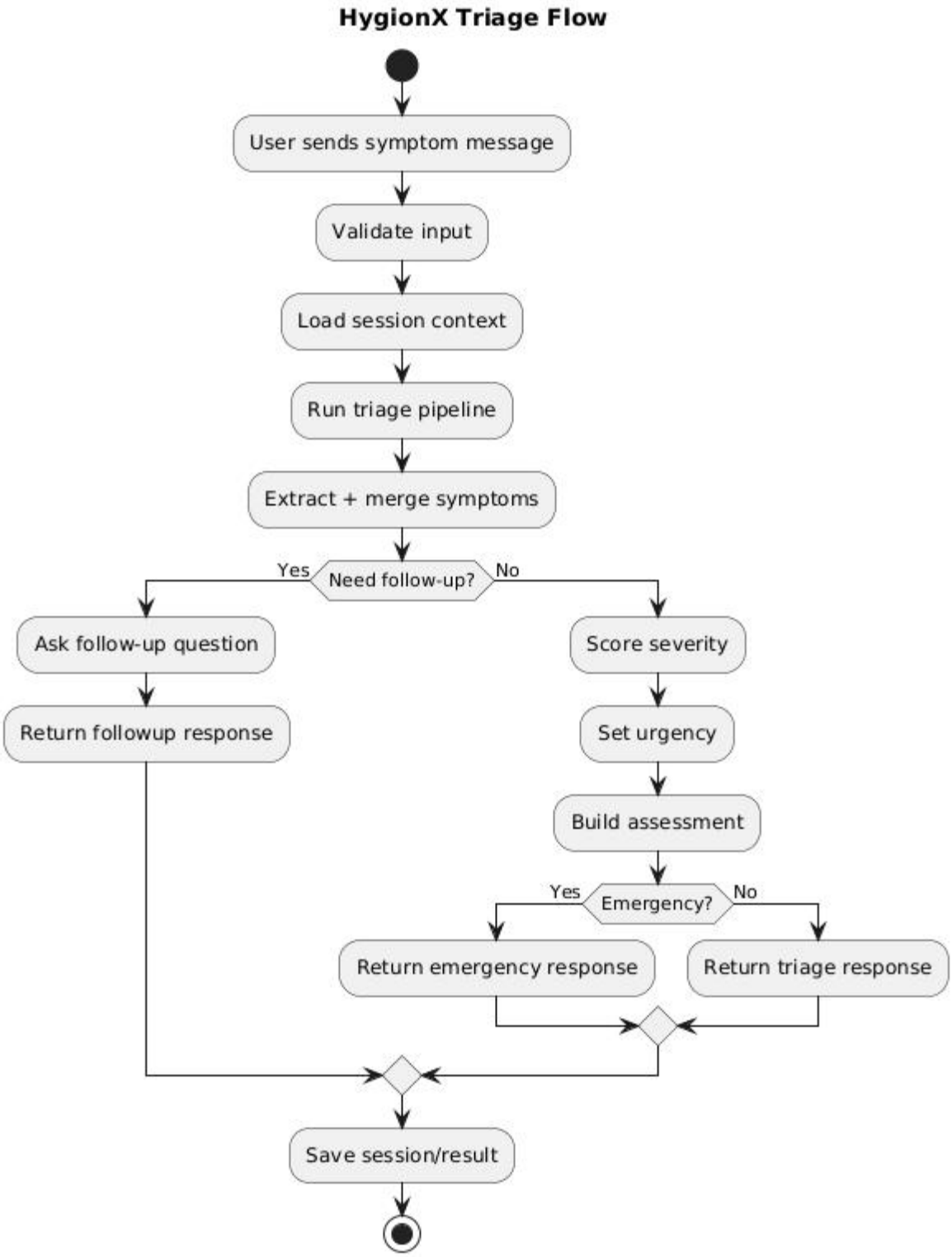

Fig 5.2.3: Activity diagram

5.2.5 Component Diagram

The component diagram of HygionX illustrates the high-level architecture of the system by showing how different software components are organized and interact with each other. It provides a clear view of the system's modular design, where each component performs a specific function. This diagram helps in understanding how the overall healthcare chatbot system is structured and how different modules collaborate to process user input and generate meaningful medical responses.

The process begins with the User, who interacts with the system through the Frontend UI (HTML/CSS/JS). This interface is responsible for collecting user inputs such as symptoms and displaying the chatbot's responses. It ensures a smooth and user-friendly experience by enabling real-time communication. The frontend sends user requests to the backend through API calls, acting as a bridge between the user and the system's internal processing modules.

The Flask Backend (app.py) acts as the central controller of the system. It receives requests from the frontend, processes them, and coordinates communication between various components. The backend directs the input to the Triage Pipeline (triage_pipeline.py), which performs initial processing and prepares the data for analysis. This pipeline then forwards the processed information to the Triage Engine (HygionXMediAITriage), where core decision-making operations such as symptom analysis, severity scoring, and recommendation generation are performed

The Triage Engine depends on the ML Model Loader (model_loader.py) to load and utilize trained machine learning models for accurate predictions. At the same time, the backend interacts with the MySQL database (via SQLAlchemy models.py) to store and retrieve user data, session details, chat history, and analysis results. This ensures data persistence and enables personalized healthcare recommendations based on past interactions. The database plays a crucial role in maintaining system reliability and continuity.

In addition to core components, the system integrates external modules such as the LLM Client (llm_client.py) and the Wiki Retriever (wiki_retriever.py). The LLM Client enhances response generation by improving language understanding and producing more natural, context-aware replies. The Wiki Retriever fetches relevant medical information from external sources to provide additional knowledge to users. Overall, the component diagram highlights a well-structured, scalable, and modular architecture that ensures efficient communication, easy maintenance, and future extensibility of the HygionX system.

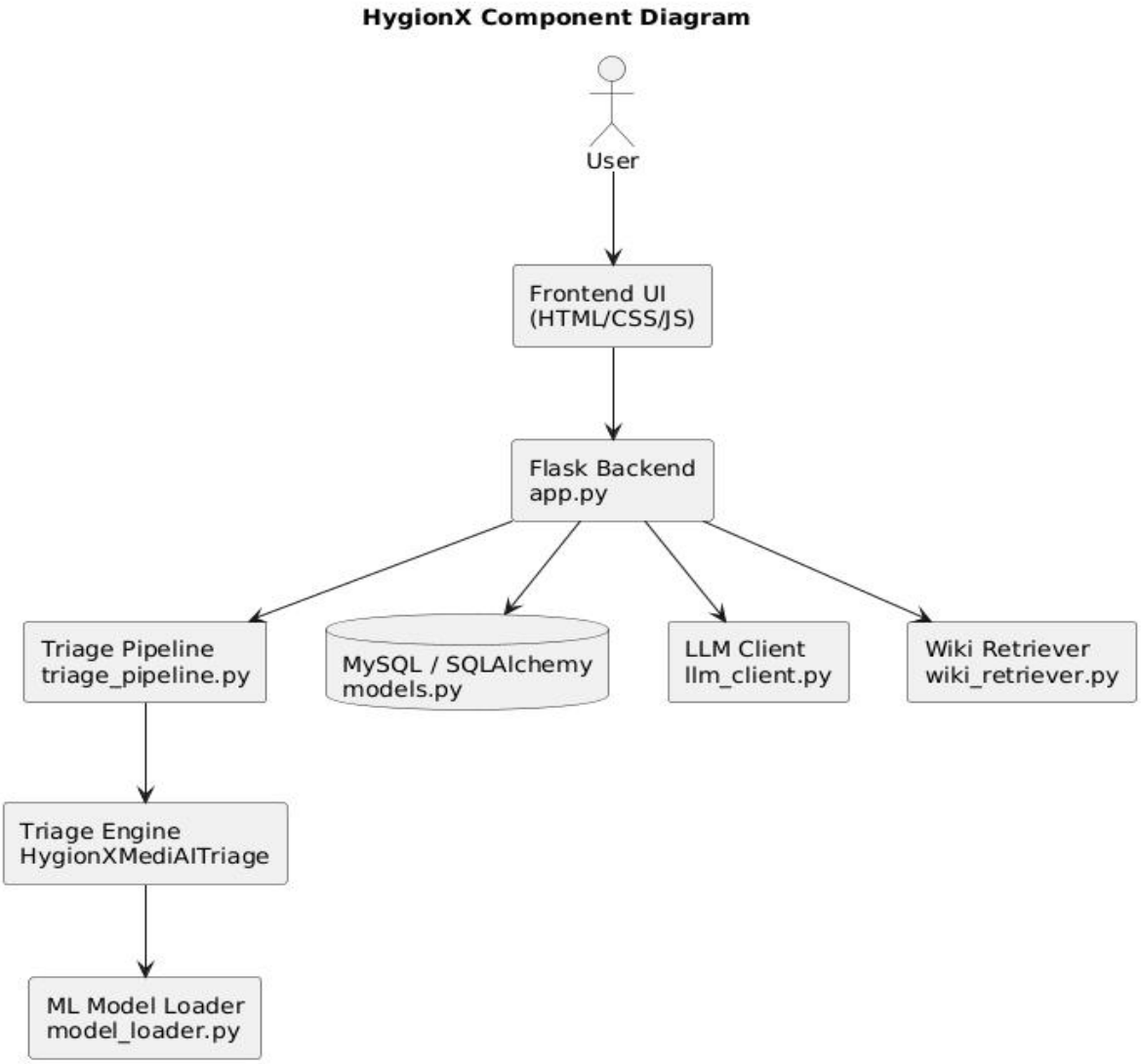

Fig 5.2.4: Component diagram

5.2.6 Deployment Diagram

The Deployment Diagram represents the physical architecture of the HygionX system, showing how software components are deployed on hardware nodes and how they communicate with each other in a real-world environment. It provides a clear understanding of where each component is located and how data flows across different parts of the system.

In the HygionX system, the deployment begins with the Client Device, which can be a mobile phone, laptop, or web browser. The user interacts with the system through the Frontend UI (HTML, CSS, JavaScript). This interface allows users to enter symptoms and view chatbot responses. The frontend communicates with the backend server using HTTP/HTTPS protocols, ensuring secure and real-time data exchange.

The Web/Application Server, implemented using Flask (Python), acts as the central controller of the system. It handles incoming requests from the frontend through API endpoints such as /api/chat. The backend manages session handling, routing, and communication between internal components. It ensures that user requests are properly processed and forwarded to the appropriate modules for further analysis.

The system also includes an AI Processing Server, which is responsible for performing intelligent analysis of user input. This server contains components such as the Triage Pipeline, Triage Engine, and ML Model Loader. These components work together to process user symptoms using NLP techniques, extract relevant information, calculate severity levels, and generate healthcare recommendations. This separation of AI processing improves performance and scalability.

The Database Server, using MySQL with SQLAlchemy, stores all essential data including user profiles, sessions, chat messages, and triage results. It ensures data persistence and enables the system to provide personalized responses based on previous interactions. The backend communicates with the database server through secure internal connections to store and retrieve information efficiently.

Additionally, the system integrates External Services such as Firebase Authentication, LLM APIs, and Wiki Retrieval APIs. These services provide authentication, enhanced response generation, and additional medical knowledge. They

are accessed via the internet and improve the overall functionality and intelligence of the system.

The communication between all these components is carried out through secure network channels, including HTTP/HTTPS for client-server communication and internal network connections for backend interactions. This ensures reliable and secure data transfer across the system.

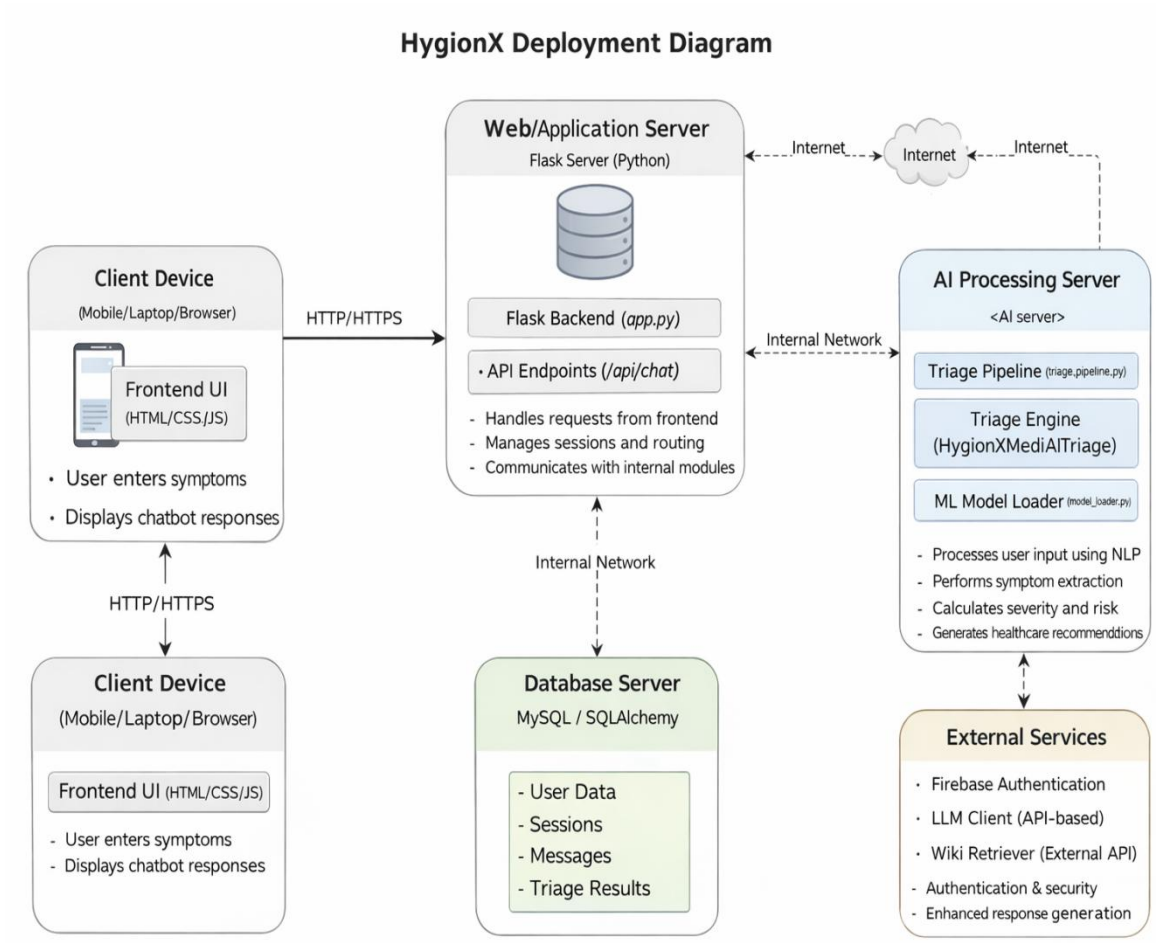

Fig 5.2.5: Deployment diagram

Chapter 6

SYSTEM IMPLEMENTATION

6.1 IMPLEMENTATION PROCESS

The system implementation process of HygionX involves the systematic development and integration of various components to build a fully functional AI-powered healthcare chatbot. It transforms the conceptual system design into a practical, real-world application by combining multiple layers such as the frontend, backend, database, and AI modules. This process ensures that each component is developed independently and then integrated seamlessly to achieve smooth communication and efficient performance.

The implementation follows a structured approach, starting from environment setup and progressing through interface design, backend logic development, database configuration, and AI model integration. Each module is carefully designed to handle specific tasks, such as user interaction, data processing, storage, and intelligent decision-making. Proper coordination between these modules ensures accurate symptom analysis, reliable response generation, and secure data handling.

Additionally, the system incorporates modern technologies such as Natural Language Processing, machine learning models, and API-based services to enhance its capabilities. The implementation process also emphasizes scalability, security, and maintainability, allowing the system to handle increasing user demands and future enhancements. Overall, this structured implementation approach ensures that HygionX operates efficiently, delivers personalized healthcare guidance, and provides a reliable and user-friendly experience.

- **Environment Setup**

The implementation begins with setting up the development environment. This includes installing tools such as Python, Flask, MySQL, and libraries like HuggingFace Transformers, SciSpacy, and PyTorch. Development platforms like VS Code and Jupyter Notebook are used for coding and testing. Proper setup ensures smooth development and compatibility between system components.

- **Frontend Development**

The frontend is developed using HTML, CSS, and JavaScript to create a user-friendly interface. It allows users to input symptoms and view chatbot responses in real time. The interface is designed to be responsive and intuitive, ensuring ease of use. It also handles communication with the backend through API calls.

- **Backend Development**

The backend is implemented using the Flask framework. It provides RESTful APIs that process user requests and return responses. The backend manages session handling, routing, and communication between different modules. It acts as the central controller of the system, ensuring smooth data flow.

- **Database Integration**

The system uses MySQL with SQLAlchemy for data storage. It stores user profiles, chat history, sessions, and triage results. The database ensures data persistence and supports personalized responses based on previous interactions. Efficient database design improves performance and reliability.

- **AI and NLP Integration**

The core functionality is implemented using AI and NLP techniques. HuggingFace transformers are used for text processing, and SciSpacy is used for symptom extraction. The triage engine analyzes the data, calculates severity levels, and generates recommendations. This module is responsible for intelligent decision-making.

- **External Services Integration**

The system integrates external services such as Firebase Authentication for secure login and APIs for knowledge retrieval and response enhancement. These services improve system functionality by adding security, scalability, and additional medical insights.

- **Testing and Validation**

After development, the system undergoes thorough testing. Different test cases are used to verify input handling, response accuracy, and system performance. Errors and bugs are identified and fixed to ensure reliability and correctness of the system.

- **Deployment**

Finally, the system is deployed on servers or cloud platforms. The frontend is made accessible to users, while the backend, database, and AI modules are hosted securely. Proper deployment ensures that the system runs efficiently in a real-world environment.

The system implementation process of HygionX ensures that all components are developed, integrated, and deployed in a coordinated and efficient manner. It brings together the frontend, backend, database, and AI modules into a unified system that functions smoothly in real-world scenarios. This integration enables seamless communication between components, ensuring that user inputs are processed accurately and responses are generated effectively.

By incorporating advanced technologies such as Natural Language Processing and machine learning, the system is able to provide accurate, personalized, and explainable healthcare support. The implementation also focuses on maintaining data security and privacy through proper authentication and secure storage mechanisms.

Furthermore, this structured approach improves system reliability by minimizing errors and ensuring consistent performance under different conditions. It also supports scalability, allowing the system to handle increasing user loads and additional features in the future. Overall, the implementation process lays a strong foundation for continuous improvement, adaptability, and long-term sustainability of the HygionX system.

6.2 MODULE IMPLEMENTATION

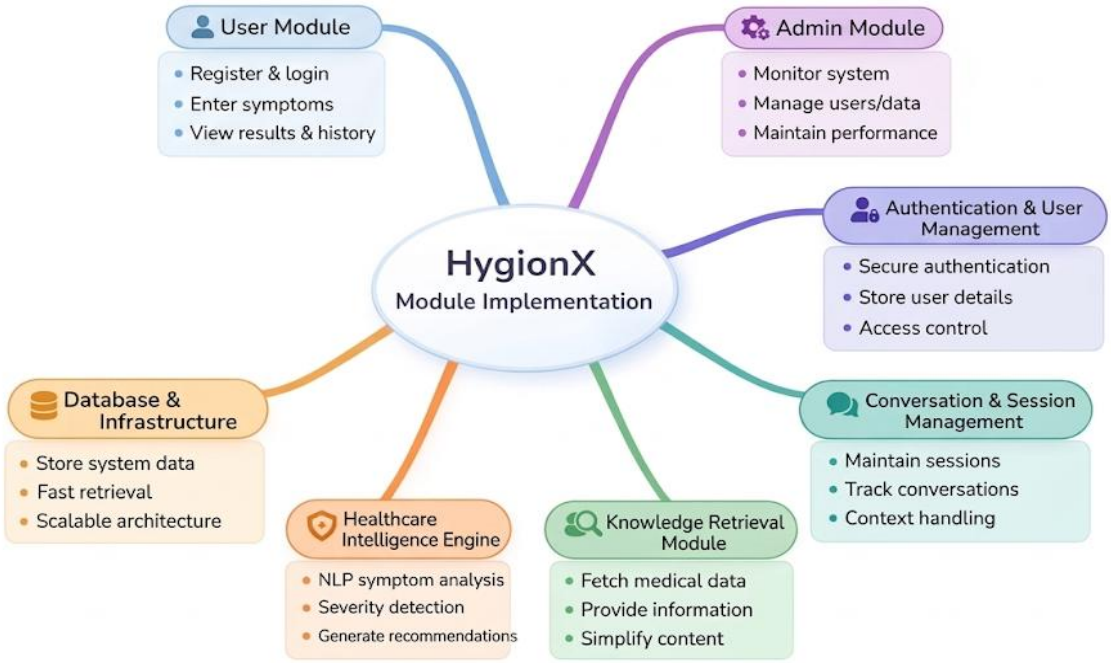

Fig 6.2.1: List of Modules

The HygionX system is implemented through several major modules, each responsible for handling a specific part of the application. These modules are designed in a structured and modular way to ensure smooth development, easy maintenance, and efficient system performance. Each module performs a dedicated function, and together they form a complete AI-powered healthcare chatbot system

- **User Module:**

The User Module enables users to interact with the HygionX system through a web-based interface. It allows users to register and log in securely to access the platform. Once authenticated, users can enter their symptoms in natural language and receive AI-generated healthcare guidance.

This module also allows users to view their previous results and interaction history, ensuring continuity and personalization. It plays a crucial role in delivering a smooth and user-friendly experience.

- **Admin Module:**

The Admin Module is responsible for managing and monitoring the overall system operations. It allows administrators to oversee user activity, manage stored data, and ensure that the system performs efficiently.

This module helps maintain system stability by monitoring performance, handling system configurations, and ensuring that all components function without disruption. It is essential for maintaining reliability and proper system management.

- **Authentication & User Management Module:**

This module handles secure authentication and user account management. It enables users to register and log in using a secure authentication system, ensuring that only authorized users can access the platform.

User details such as personal information and medical data are stored securely, and proper access control mechanisms are implemented. This module enhances both security and personalization within the system.

- **Conversation & Session Management Module:**

The Conversation & Session Management Module manages the interaction flow between the user and the chatbot. It maintains session continuity by storing chat history and tracking ongoing conversations.

This module ensures that context is preserved across multiple interactions, allowing the system to provide meaningful follow-up questions and accurate responses. It plays a key role in improving the consistency and intelligence of the chatbot.

- **Knowledge Retrieval Module:**

The Knowledge Retrieval Module is responsible for fetching medical information from external sources such as medical databases or APIs. It provides users with relevant and accurate information related to diseases and conditions.

The module presents this information in a simplified and easy-to-understand format, helping users gain better insights into their health.

- **Healthcare Intelligence Engine:**

The Healthcare Intelligence Engine is the core component of the HygionX system. It processes user input using Natural Language Processing (NLP) techniques to extract symptoms and relevant medical information.

This module analyzes symptom severity, determines risk levels, and generates appropriate recommendations. It also provides explainable outputs, including reasoning and confidence scores, which improve user trust and system reliability.

- **Database & Infrastructure Module:**

The Database & Infrastructure Module is responsible for storing and managing all system data. It maintains user profiles, chat history, detected symptoms, and triage results in a structured database.

This module ensures efficient data retrieval, data consistency, and long-term storage. The infrastructure is designed to be scalable and reliable, supporting smooth system performance and future expansion.

The module implementation of HygionX provides a well-structured approach to building a complete AI-powered healthcare chatbot system. By dividing the system into distinct modules such as user interaction, administration, authentication, session management, knowledge retrieval, intelligence processing, and data storage, each component can be developed and managed efficiently.

Furthermore, the integration of all modules ensures seamless communication between system components, enabling accurate data processing and real-time response generation. This modular architecture improves scalability, flexibility, and maintainability, making the system suitable for real-world deployment and future enhancements.

Chapter 7
SYSTEM STUDY & TESTING

7.1 FEASIBILITY STUDY

The feasibility of the HygionX system is analyzed to ensure that the proposed AI-based medical triage application is practical, efficient, and beneficial for users. This study evaluates whether the system can be developed and deployed without excessive resource demands while delivering meaningful healthcare guidance.

The feasibility analysis considers the following key aspects:

1. Technical Feasibility
2. Social Feasibility
3. Economical Feasibility

1. Technical Feasibility

This study evaluates whether the required technology and resources are available to develop and run the system efficiently.

The HygionX system is technically feasible as it is built using widely adopted and scalable technologies such as Flask, Firebase Authentication, SQLAlchemy, and machine learning components.

- The system supports both ML-assisted mode and fallback rule-based mode, ensuring reliability even when advanced models are unavailable.
- It does not require high-end hardware and can run on standard web servers.
- The modular architecture enables smooth integration between frontend, backend, and AI components.
- The system is designed to be scalable and maintainable for future enhancements.

Thus, the system has moderate technical requirements and can be implemented effectively.

2. Social Feasibility

This aspect evaluates the level of acceptance of the system among users. HygionX is designed to be user-friendly and accessible, ensuring high acceptance among users. The system can be easily accessed through standard web browsers without requiring installation or technical expertise, making it convenient for a wide range of users.

- The chat-based interface allows users to describe symptoms naturally.
- The system provides clear explanations, risk levels, and recommendations, improving user trust.
- Safety disclaimers are included to ensure users understand that it is not a replacement for professional medical advice.
- Minimal training is required due to the intuitive interface.

Hence, the system is socially acceptable and easy to adopt.

3. Economic Feasibility

This study evaluates the cost-effectiveness of the system. The HygionX system is economically feasible as it utilizes mostly open-source and freely available technologies. Additionally, the system requires minimal maintenance and operational costs, making it financially sustainable in the long run.

- Technologies such as Flask and Python libraries are open-source.
- Firebase provides scalable authentication with a free tier.
- Deployment and maintenance costs are relatively low.
- The system can scale gradually without significant additional cost.
-

Additionally, the system provides indirect economic benefits:

- Reduces unnecessary hospital visits for minor conditions
- Saves time for both users and healthcare providers
- Provides quick preliminary guidance, improving efficiency

Thus, the system is cost-effective and financially viable.

7.2 SOFTWARE TESTING

The purpose of testing is to identify errors and ensure that the HygionX system functions correctly, efficiently, and safely. Testing verifies that the system meets user requirements and provides reliable medical guidance. In addition to validating correctness, testing in HygionX focuses on ensuring continuity of conversation, accurate symptom interpretation, proper urgency classification, and smooth interaction between system components.

There are various types of tests. Each type of test address a specific testing requirement.

Types of Testing:

1. Unit Testing
2. IntegrationTesting
3. System Testing
4. Performance Testing
5. Acceptance Testing

1. Unit Testing:

Unit testing involves verifying individual components of the system in isolation to ensure that each module performs its intended function correctly. This type of testing is carried out during the development phase and focuses on validating internal logic and data processing.

In HygionX, unit testing is applied to critical components such as:

- Symptom extraction modules, including both machine learning-based and rule-based fallback approaches
- Severity scoring algorithms that compute risk levels based on detected symptoms
- Emergency detection logic that identifies high-risk conditions
- API endpoints and backend utility functions

Each unit is tested with multiple input scenarios, including valid, invalid, and edge-case inputs, to ensure robustness. Unit testing helps detect errors early, reduces debugging effort in later stages, and ensures a strong foundation for integration.

Test Case ID	Input	Function	Expected Output	Result
UT-01	"I am suffering from fever and headache"	Symptom Extraction	["fever", "headache"]	Passed
UT-02	Symptoms list	Severity Scoring	Score between 1–10	Passed
UT-03	Chest pain	Urgency Detection	"Emergency"	Passed
UT-04	API Health request	EndpointValidation	Status OK	Passed

Table 7.2.1 : Unit Testing

2. Integration Testing:

Integration testing evaluates the interaction between different modules of the system to ensure that they work together seamlessly. While individual components may function correctly on their own, integration testing verifies that combined modules exchange data properly and produce correct results.

In HygionX, integration testing focuses on:

- Communication between the frontend user interface and backend APIs
- Interaction between the chat API and the triage engine
- Authentication and session management using Firebase

This testing ensures that user inputs are correctly transmitted, processed, and stored, and the responses generated by system are accurate and consistent across modules.

Test Case ID	Module Interaction	Input	Expected Output	Result
IT-01	Chat UI => API	User message	Valid structured response returned	Passed
IT-02	API =>Triage Engine	Symptoms text	Follow-up or final assessment generated	Passed
IT-03	Triage Engine => Database	Final result	Stored successfully	Passed
IT-04	Authentication => Backend	Valid Token	User Authenticated Successfully	Passed

Table 7.2.2 : Integration Testing

3. System Testing:

System testing evaluates the complete HygionX application as a fully integrated system. It ensures that all components work together correctly and that the system behaves as expected in real-world scenarios.

This testing phase includes:

- End-to-end validation of the chat-based triage workflow
- Verification of multi-turn conversations and context retention
- Testing of emergency detection and escalation mechanisms
- Evaluation of system responses for both typical and edge-case inputs
- Verification of user interface behavior and clarity of displayed results

System testing simulates real user interactions and helps identify issues related to overall system functionality, usability, and stability.

Test Case ID	Scenario Description	Input Provided	Expected System Behaviour	Result
ST-01	User enters multiple symptoms in a single message	"I have fever, headache and nausea"	System extracts all symptoms and generates relevant follow-up questions	Passed
ST-02	User provides emergency-related symptoms	"I have severe chest pain and difficulty breathing"	System immediately classifies as emergency and displays urgent medical advice	Passed
ST-03	User enters mild/common symptoms	"I have a mild cough and cold"	System classifies as low/moderate risk and provides guidance	Passed
ST-04	User provides incomplete or vague input	"I feel unwell"	System asks clarification questions to gather more details	Passed
ST-05	User enters invalid or unrelated input	Random text or empty input	System handles without crashing and prompts user again	Passed
ST-06	Multi-turn conversation scenario	Sequential symptom inputs across messages	System retains previous context and refines assessment	Passed

Table 7.2.3 : System Testing

4. Performance Testing:

Performance testing evaluates the responsiveness, scalability, and stability of the HygionX system under varying workloads and conditions. Since the system is developed in a local environment and not deployed for real-world usage, performance testing is conducted by simulating concurrent user interactions.

In HygionX, performance testing includes:

- Measuring response time of the chat API for different types of inputs
- Simulating multiple concurrent requests using browser sessions and repeated API calls
- Evaluating system behavior under continuous and rapid request conditions
- Identifying potential bottlenecks in processing or data handling

Performance testing was conducted in a controlled local environment by simulating concurrent usage through multiple browser instances and repeated requests to the backend. This approach helps ensure that the system remains responsive and stable even under increased load conditions.

Test Case ID	Scenario	Input Provided	Expected Output	Result
PT-01	Simulated concurrent requests	Multiple parallel requests to chat API	Response time < 2 seconds	Passed
PT-02	Continuous request handling	Repeated API calls over time	No system crash or performance degradation	Passed
PT-03	Large input processing	Long symptom descriptions	System processes input without delay or failure	Passed
PT-04	LLM response handling	Complex symptom input requiring detailed analysis	System generates response within acceptable time without timeout	Passed

Table 7.2.4 : Performance Testing

5. Acceptance Testing:

Acceptance testing is the final phase of testing, where the system is evaluated from the user's perspective to determine whether it meets the specified requirements and expectations. This testing ensures that the system is ready for deployment.

In HygionX, acceptance testing focuses on:

- Usability and ease of interaction with the chat interface
- Clarity and usefulness of the medical guidance provided
- Accuracy of symptom-based triage results
- Overall user satisfaction and experience

This phase may involve feedback from test users, helping to identify any remaining issues or improvements before final deployment.

Test Case ID	Criteria	Expected Behaviour	Result
AT-01	Basic symptom input	Clear triage result displayed	Passed
AT-02	Follow-up interaction	Logical questions asked	Passed
AT-03	Emergency case	Immediate warning displayed	Passed
AT-04	UI clarity	Results displayed clearly	Passed
AT-05	User experience	Smooth and intuitive interaction	Passed
AT-06	Display of confidence score	System displays confidence score clearly along with assessment	Passed

Table 7.2.5 : Acceptance Testing

Chapter 8

SOURCE CODE

Disease Prediction Model

```
# Import the libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import MultiLabelBinarizer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from xgboost import XGBClassifier
# Load the dataset
df=pd.read_csv(r"C:\Users\chips\Desktop\HygionXModel Files\Datasets\model_ds.csv")
df.head()
# Remove unknown cases
df = df[df["disease"] != "unknown"]
df.shape
# Plot Disease Distribution
plt.figure(figsize=(14,6))
disease_counts = df["disease"].value_counts().head(20)
sns.barplot(x=disease_counts.values, y=disease_counts.index)
plt.title("Top 20 Diseases in Dataset")
plt.xlabel("Number of Cases")
plt.ylabel("Disease")
plt.show()
# Symptom count Distribution
plt.figure(figsize=(6,4))
sns.countplot(x="symptom_count", data=df)
```

```
plt.title("Number of Symptoms per Patient")
plt.xlabel("Symptom Count")
plt.ylabel("Number of Patients")
plt.show()
# Convert symptoms to List
df["symptom_list"] = df["symptoms"].apply(
    lambda x: sorted(x.split(";"))
)
# Convert to Binary Features
mlb = MultiLabelBinarizer()
symptom_matrix = mlb.fit_transform(df["symptom_list"])
symptom_df = pd.DataFrame(symptom_matrix, columns=mlb.classes_)
# Features and Target
X = symptom_df
y = df["disease"]
# Label Encoding
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y_encoded = le.fit_transform(y)
# Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.2,
    stratify=y_encoded,
    random_state=42
)
# Train XGBoost Model
model = XGBClassifier(
    objective="multi:softprob",
```

```
    eval_metric="mlogloss",
    n_estimators=400,
    max_depth=6,
    learning_rate=0.05,
    subsample=0.9,
    colsample_bytree=0.9,
    random_state=42
)
model.fit(
    X_train,
    y_train,
    eval_set=[(X_test, y_test)],
    verbose=False
)
# Saving the Disease Prediction Model
import joblib
joblib.dump(model,r"C:\Users\chips\Desktop\HygionXModelFiles\Models\m14models\d
                    isease_model.pkl")
# Saving the Label Encoder Model
joblib.dump(le,r"C:\Users\chips\Desktop\HygionXModelFiles\Models\m14models\diseas
                    e_label_encoder.pkl")
# Saving the MultiLabelBinarizer
joblib.dump(mlb,r"C:\Users\chips\Desktop\HygionXModelFiles\Models\m14models\sym
                    ptom_vectorizer.pkl")
# Prediction
pred = model.predict(X_test)
pred_labels = le.inverse_transform(pred)
# Accuracy Measurement
print("Accuracy:", accuracy_score(y_test, pred))
# Feature Importance Plot
```

```
importances = model.feature_importances_
indices = np.argsort(importances)[-15:]
plt.figure(figsize=(6,4))
plt.barh(range(len(indices)), importances[indices])
plt.yticks(range(len(indices)), X.columns[indices])
plt.title("Top 15 Important Symptoms for Disease Prediction")
plt.show()
# Top 3 Disease Prediction
probs = model.predict_proba(X_test)
top3 = np.argsort(probs, axis=1)[:, -3:][:, ::-1]
for i in range(5):
    print("Actual:", le.inverse_transform([y_test[i]])[0])
    print("Top3:", le.inverse_transform(top3[i]))
# Top 3 Accuracy
top3_preds = top3
top3_accuracy = np.mean(
    [y_test[i] in top3[i] for i in range(len(y_test))]
)
print("Top-3 Accuracy:", top3_accuracy)
# Confusion Matrix
from sklearn.metrics import confusion_matrix
import seaborn as sns
cm = confusion_matrix(y_test, pred)
plt.figure(figsize=(6,4))
sns.heatmap(cm, cmap="Blues")
plt.title("Disease Prediction Confusion Matrix")
plt.show()
# Test cases
import joblib
import numpy as np
```

```
model = joblib.load(r"C:\Users\chips\Desktop\HygionXModelFiles\Models\m14
                    models\disease_model.pkl")
le=joblib.load(r"C:\Users\chips\Desktop\HygionXModelFiles\Models\m14
                    models\disease_label_encoder.pkl")
mlb = joblib.load(r"C:\Users\chips\Desktop\HygionXModelFiles\Models\m14
                    models\symptom_vectorizer.pkl")
tests = [
(["fever","cough","fatigue"], "Influenza"),
(["fever","headache","chills"], "Malaria"),
(["fever","rapid breathing","confusion"], "Sepsis"),
(["cough","shortness of breath","fatigue"], "Bronchitis"),
(["sneezing","runny nose","nasal congestion"], "Common Cold"),
(["headache","nausea","sensitivity to light"], "Migraine"),
(["skin rash","itching skin","skin redness"], "Dermatitis"),
]
correct_top1 = 0
correct_top3 = 0
for symptoms, actual in tests:
    X = mlb.transform([symptoms])
    probs = model.predict_proba(X)[0]
    top3_idx = np.argsort(probs)[-3:][::-1]
    preds = le.inverse_transform(top3_idx)
    print(symptoms, "->", preds, "| actual:", actual)
    if preds[0] == actual:

        correct_top1 += 1
    if actual in preds:
        correct_top3 += 1
print("\nTop-1 Accuracy:", correct_top1 / len(tests))
print("Top-3 Accuracy:", correct_top3 / len(tests))
```

Triage Model

```
# Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import MultiLabelBinarizer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
# Load Dataset
df = pd.read_csv("/content/synthetic_patient_dataset (1).csv")
df.head()
# Basic Dataset Info
print("Dataset shape:", df.shape)
df.info()
# Check missing values
df.isnull().sum()
# Triage Distribution Visualization
plt.figure(figsize=(6,4))
sns.countplot(x="triage", data=df)
plt.title("Triage Level Distribution")
plt.xlabel("Triage Level")
plt.ylabel("Count")
plt.show()
# Severity score Distribution
plt.figure(figsize=(6,4))
sns.histplot(df["severity_score"], bins=20)
plt.title("Severity Score Distribution")
plt.show()
```

```
# Symptom count Distribution
plt.figure(figsize=(6,4))
sns.countplot(x="symptom_count", data=df)
plt.title("Number of Symptoms per Patient")
plt.show()
# Convert symptoms to List
df["symptom_list"] = df["symptoms"].apply(lambda x: x.split(";"))
df.head()
# Convert symptoms to Binary Features
mlb = MultiLabelBinarizer()
symptom_matrix = mlb.fit_transform(df["symptom_list"])
symptom_df = pd.DataFrame(symptom_matrix, columns=mlb.classes_)
# Combine features
X = pd.concat([
    symptom_df,
    df[["severity_score"]]
], axis=1)
y=df["triage"]
# Train/Test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
# Train Random Forest
model = RandomForestClassifier(
    n_estimators=200,
    max_depth=12,
    min_samples_split=10,
    min_samples_leaf=5,
```

```
    random_state=42
)
model.fit(X_train, y_train)
# Predictions
pred = model.predict(X_test)
# Accuracy
print("Accuracy:", accuracy_score(y_test, pred))
# Classification report
print(classification_report(y_test, pred))
# Confusion Matrix
cm = confusion_matrix(y_test, pred)
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt="d",
            xticklabels=model.classes_,
            yticklabels=model.classes_)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Triage Confusion Matrix")
plt.show()
# Saving the Model
import joblib
joblib.dump(model, "triage_model.pkl")
# Saving the symptom Encoder
joblib.dump(mlb,"symptom_encoder.pkl")
# Saving Feature column order
joblib.dump(X.columns.tolist(), "triage_features.pkl")
# Prediction Cases
def predict_triage_probability(symptoms, severity_score):
    if isinstance(symptoms, str):
        symptoms = symptoms.split(";")
```

```
    symptom_vector = mlb.transform([symptoms])
    symptom_df_test = pd.DataFrame(symptom_vector, columns=mlb.classes_)
    symptom_df_test["severity_score"] = severity_score
    pred = model.predict(symptom_df_test)[0]
    probs = model.predict_proba(symptom_df_test)[0]
    prob_dict = {k: float(v) for k,v in zip(model.classes_, probs)}
    return pred, prob_dict
# Prediction Test case - 1
symptoms = "cough;runny nose;fatigue"
severity_score = 4
result = predict_triage_probability(symptoms, severity_score)
print("Symptoms:", symptoms)
print("Predicted Triage:", result)
# Prediction Test case - 2
symptoms = "fever;cough;fatigue;headache"
severity_score = 7
result = predict_triage_probability(symptoms, severity_score)
print("Symptoms:", symptoms)
print("Predicted Triage:", result)
# Prediction Test case - 3
symptoms = "chest pain;shortness of breath;fatigue"
severity_score = 11
result = predict_triage_probability(symptoms, severity_score)
print("Symptoms:", symptoms)
print("Predicted Triage:", result)
print("Predicted Triage:", result)
symptoms = "chest pain;shortness of breath"
predict_triage_probability(symptoms, 12)
```

app.py

```
import os
import json
from dotenv import load_dotenv
from flask import Flask, jsonify, request
from triage_pipeline import run_triage_pipeline, hygionx_triage
load_dotenv()
MODELS_AVAILABLE = False
try:
    from models import db, init_app, get_database_url, User, Session, Message, SymptomsDetected, TriageResult
    MODELS_AVAILABLE = True
except ImportError:
    db = init_app = get_database_url = None
    User = Session = Message = SymptomsDetected = TriageResult = None
FIREBASE_AVAILABLE = False
try:
    import firebase_admin
    from firebase_admin import auth as firebase_auth, credentials
    FIREBASE_AVAILABLE = True
except ImportError:
    firebase_admin = firebase_auth = credentials = None
app = Flask(__name__)
app.config["SECRET_KEY"] = os.environ.get("FLASK_SECRET_KEY", "hygionx-dev-secret-key")
_firebase_initialized = False
if FIREBASE_AVAILABLE and not firebase_admin._apps:
    cred_path = os.environ.get("FIREBASE_CREDENTIALS_PATH")
    if cred_path and Path(cred_path).exists():
        firebase_admin.initialize_app(credentials.Certificate(cred_path))
```

```
        _firebase_initialized = True
def _verify_firebase_token(id_token):
    if not FIREBASE_AVAILABLE or not _firebase_initialized or not id_token:
        return None
    try:
        return firebase_auth.verify_id_token(id_token)
    except Exception:
        return None
def _get_or_create_user(firebase_uid, email="", name=""):
    if not MODELS_AVAILABLE:
        return None
    user = User.query.filter_by(firebase_uid=firebase_uid).first()
    if user:
        if email:
            user.email = email
        if name:
            user.name = name
        return user
    user = User(firebase_uid=firebase_uid, email=email, name=name)
    db.session.add(user)
    db.session.flush()
    return user
def _get_or_create_session(user_id, session_id=None):
    if not MODELS_AVAILABLE:
        return None
    if session_id:
        existing = Session.query.filter_by(id=session_id, user_id=user_id).first()
        if existing:
            return existing
    session = Session(user_id=user_id)
```

```
    db.session.add(session)
    db.session.flush()
    return session
def _severity_to_urgency(score):
    if score is None:
        return "Low Risk"
    if score >= 7:
        return "High Risk"
    if score >= 5:
        return "Moderate Risk"
    return "Low Risk"
def _normalize_possible_conditions(raw_conditions):
    normalized = []
    for item in raw_conditions or []:
        if isinstance(item, dict):
            condition = (item.get("condition") or "").strip()
            probability = item.get("probability", 0)
        else:
            condition = str(item or "").strip()
            probability = 0
        if not condition:
            continue
        try:
            probability = int(round(float(str(probability).replace("%", "").strip())))
        except Exception:
            probability = 0
        normalized.append({"condition": condition, "probability": max(0, min(100,
probability))})
    return normalized
def _build_summary(symptoms, urgency, recommended_action, possible_conditions):
```

```
    symptom_text = ", ".join(symptoms[:4]) if symptoms else "general symptoms"
    condition_text = ", ".join([c["condition"] for c in possible_conditions[:2]]) if
possible_conditions else ""
    lines = [f"- Symptoms: {symptom_text}", f"- Urgency: {urgency}"]
    if condition_text:
        lines.append(f"- Possible patterns: {condition_text}")
    if recommended_action:
        lines.append(f"- Next step: {recommended_action}")
    return "\n".join(lines[:4])
if MODELS_AVAILABLE and get_database_url():
    try:
        init_app(app)
    except Exception as exc:
        MODELS_AVAILABLE = False
        print("Database unavailable:", exc)
@app.route("/api/chat", methods=["POST"])
def api_chat():
    data = request.get_json() or {}
    text = (data.get("message") or data.get("text") or "").strip()
    if not text:
        return jsonify({"error": "message or text is required"}), 400
    id_token = (data.get("idToken") or "").strip()
    if not id_token and request.headers.get("Authorization", "").startswith("Bearer "):
        id_token = request.headers["Authorization"].split(" ", 1)[1].strip()
    user = None
    session = None
    previous_messages = []
    result = None
    if MODELS_AVAILABLE:
        try:
```

```
        claims = _verify_firebase_token(id_token) if id_token else None
        if claims:
            user = _get_or_create_user(
                claims.get("uid"),
                email=claims.get("email", ""),
                name=claims.get("name", ""),
            )
            db.session.commit()
        if user:
            session = _get_or_create_session(user.id, data.get("session_id"))
        else:
            guest = User.query.filter_by(firebase_uid="default_system_user").first()
            if not guest:
                guest = User(firebase_uid="default_system_user", email="system@hygionx
                    .com", name="System User")
                db.session.add(guest)
                db.session.flush()
            session = _get_or_create_session(guest.id, data.get("session_id"))
        db.session.add(Message(session_id=session.id, role="user", content=text))
        db.session.flush()
        message_rows = (
            Message.query.filter_by(session_id=session.id)
            .order_by(Message.created_at.desc())
            .limit(10)
            .all())
        previous_messages = [{"role": row.role, "content": row.content} for row in
reversed(message_rows)]
        result = run_triage_pipeline(
            text, session_id=session_id,
            previous_messages=previous_messages,
```

```
            followup_count=data.get("followup_count", 0),
            previous_symptoms=data.get("previous_symptoms", []),
            force_triage=data.get("force_triage", False),
            known_conditions=getattr(user, "known_conditions", "") if user else "" )
        assistant_text = result.get("message") or result.get("explanation")
        if result.get("type") == "followup":
            questions = result.get("questions", [])
            assistant_text = questions[0] if isinstance(questions, list) and questions else
assistant_text
        db.session.add(Message(session_id=session.id,role="ast",content=assistant_text))
        symptoms = [s for s in result.get("symptoms", []) if s]
        severity = result.get("severity") or result.get("severity_score")
        precautions = result.get("precautions", [])
        possible_diseases = result.get("possible_diseases", [])
        if symptoms:
            db.session.add(
                SymptomsDetected(
                    session_id=session.id,
                    symptoms_json=json.dumps(symptoms),
                    severity_score=severity,
                    precautions_json=json.dumps(precautions) if precautions else None,
                    possible_diseases_json=json.dumps(possible_diseases)                    if
possible_diseases else None,
                    confidence=result.get("confidence"),
                ))
        if result.get("type") in ("triage", "emergency"):
            db.session.add(
                TriageResult(
                    session_id=session.id,
                    user_id=session.user_id,
```

```
                severity_score=severity,
                risk_level=_severity_to_urgency(severity),
                recommended_action=result.get("recommended_action")
                confidence=result.get("confidence"),
            ) )
        db.session.commit()
    except Exception as exc:
        db.session.rollback()
        print("Database error in /api/chat:", exc)
        session = None
        result = run_triage_pipeline(text)
if result is None:
    result = run_triage_pipeline(text)
session_id_out = session.id if session else None
severity = result.get("severity") or result.get("severity_score", 0)
symptoms = result.get("symptoms", [])
urgency = result.get("urgency_level") or _severity_to_urgency(severity)
confidence_val = result.get("confidence", 0.5) or 0.0
possible_conditions = _normalize_possible_conditions(
    result.get("possible_conditions_structured") or result.get("possible_conditions")or [])
recommended_action = result.get("recommended_action") or "Monitor symptoms and
seek care if they worsen."
payload = {
    "type": result.get("type", "triage"),
    "session_id": session_id_out,
    "symptoms": symptoms,
    "severity_score": severity,
    "urgency_level": urgency,
    "confidence": confidence_val,
    "assessment": result.get("explanation") or result.get("message", ""),
```

```
        "recommended_action": recommended_action,
        "possible_conditions": possible_conditions,
        "session_summary": _build_summary(symptoms, urgency, recommended_action, possible_conditions),
        "confidence_explanation": _build_confidence_explanation(
            confidence_val, symptoms, result.get("followup_count", 0)),
        "safety_message": "This AI provides guidance only and is not a substitute for professional diagnosis." }
    if result.get("type") == "followup":
        questions = result.get("questions", [])
        payload["question"] = questions[0] if isinstance(questions, list) and questions else "Can you describe it more?"
        payload["response_type"] = "question"
    else:
        payload["response_type"] = "final"
    return jsonify(payload)
@app.route("/api/hygionx/chat", methods=["POST"])
def api_triage_chat():
    body = request.get_json() or {}
    text = (body.get("message") or "").strip()
    if not text:
        return jsonify({"error": "message is required"}), 400
    result = run_triage_pipeline(text)
    if result.get("type") == "followup":
        return jsonify({"type": "followup", "questions": result.get("questions", [])})
    severity = result.get("severity") or result.get("severity_score", 0)
    return jsonify(
        {
            "type": "triage","symptoms": result.get("symptoms", []),
            "severity": severity,   "response": result.get("explanation", ""),
```

```
        "urgency": "low" if severity < 5 else "moderate" if severity < 7 else "high",
        "confidence": result.get("confidence", 0.5),
        "redFlag": severity >= 7, })
@app.route("/api/disease-info", methods=["POST"])
def api_disease_info():
    body = request.get_json() or {}
    disease_name = (body.get("disease_name") or "").strip()
    if not disease_name:
        return jsonify({"success": False, "error": "disease_name is required"}), 400
    try:
        from wiki_retriever import WikiRetriever
        retriever = WikiRetriever()
information=retriever.retrieve(disease_name,max_sentences=int(body.get("max_sentence
s", 5)))
        return jsonify(
            {
                "success": True,
                "disease_name": disease_name,
                "information": information,
                "source": "Wikipedia",
            }
        )
    except Exception:
        return jsonify({
                "success": False,
                "error": "Disease lookup is not available.",
                "disease_name": disease_name,
            }), 503
if __name__ == "__main__":
    app.run(debug=True, port=5000, use_reloader=False)
```

Chapter 9
SCREEN LAYOUTS

9.1 INTERFACE LAYOUT

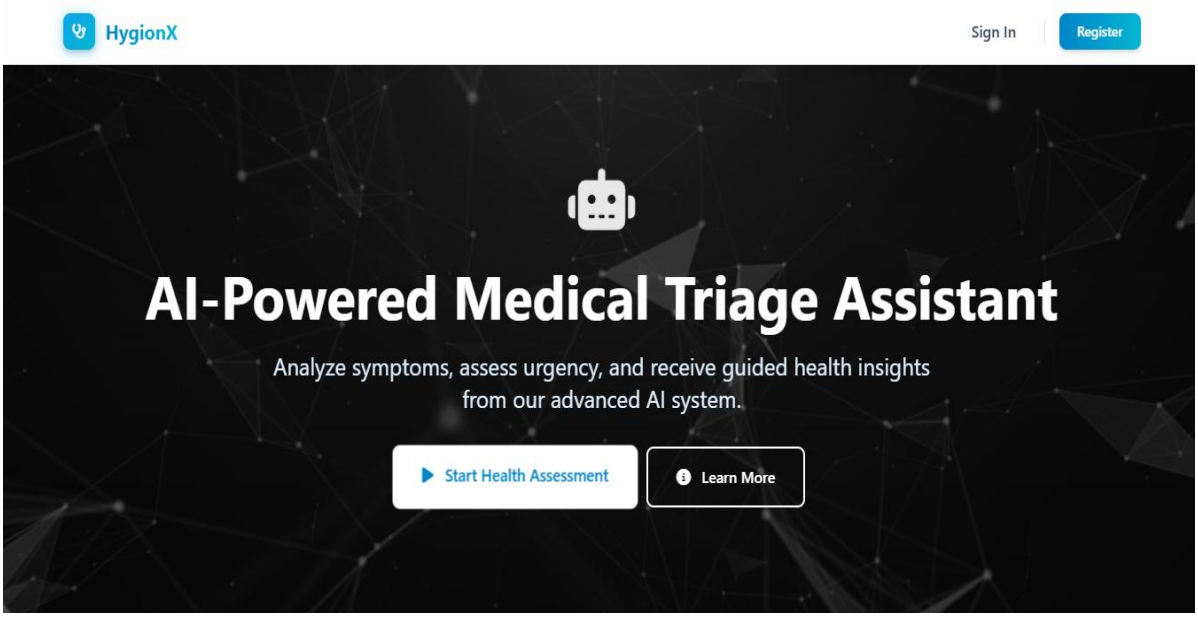

Fig 9.1.1 : Landing Page Interface

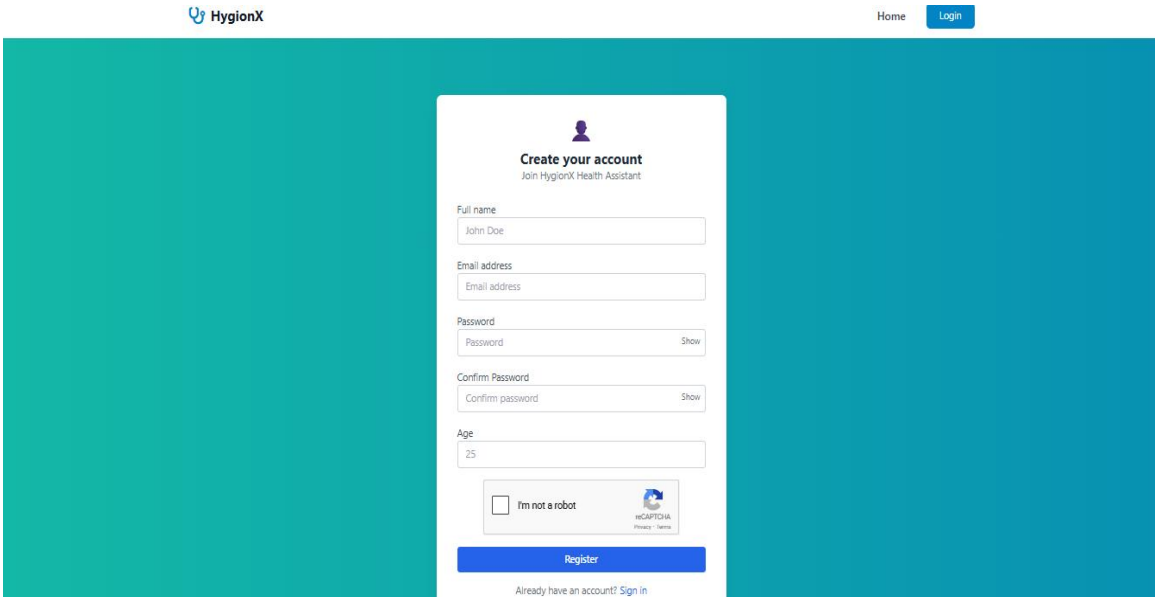

Fig 9.1.2 : Authentication Interface

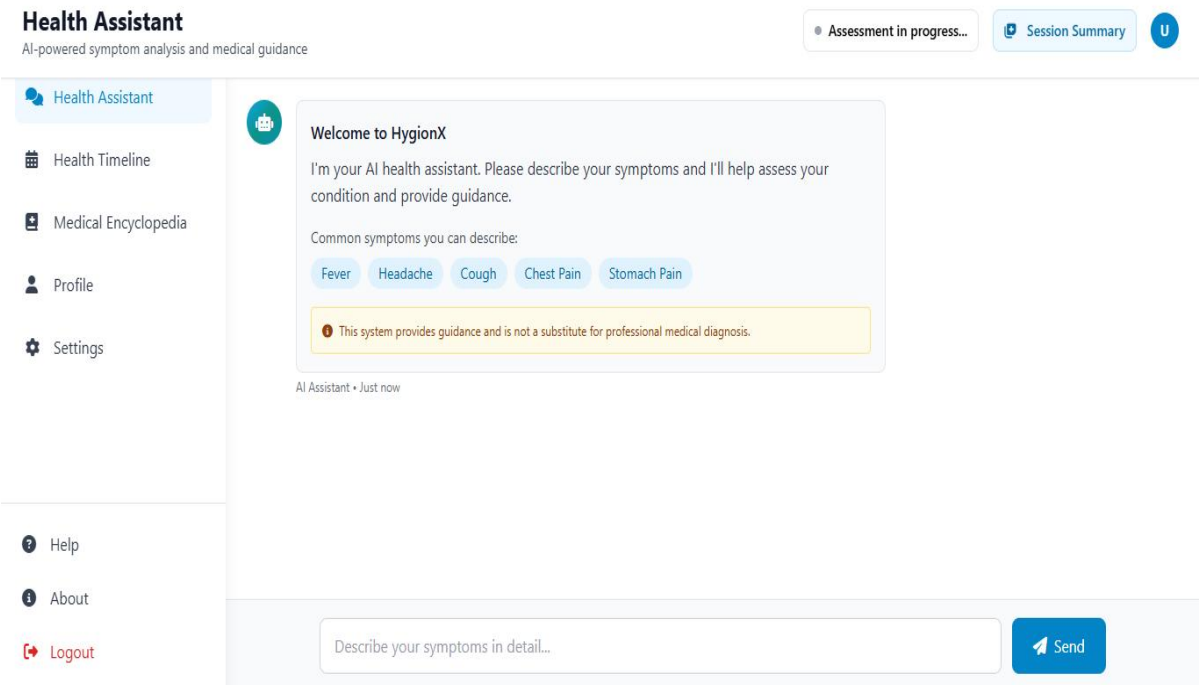

Fig 9.1.3 : Health Assistant Interface

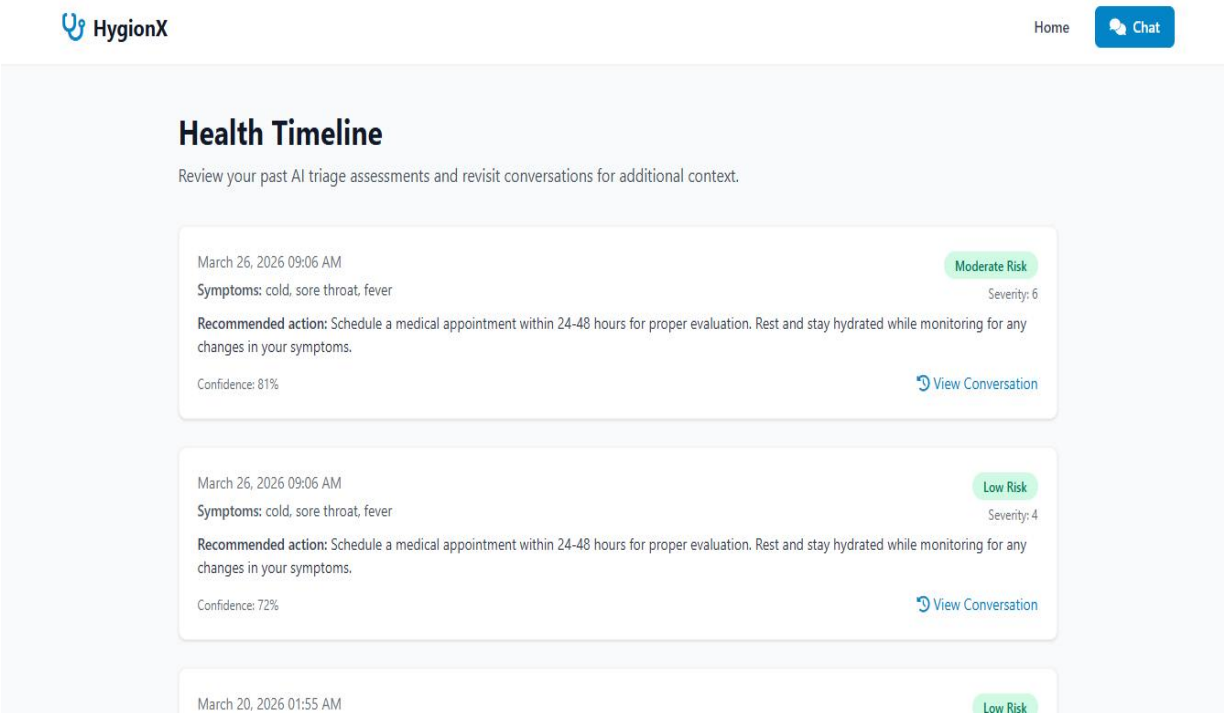

Fig 9.1.4 : Health Timeline Interface

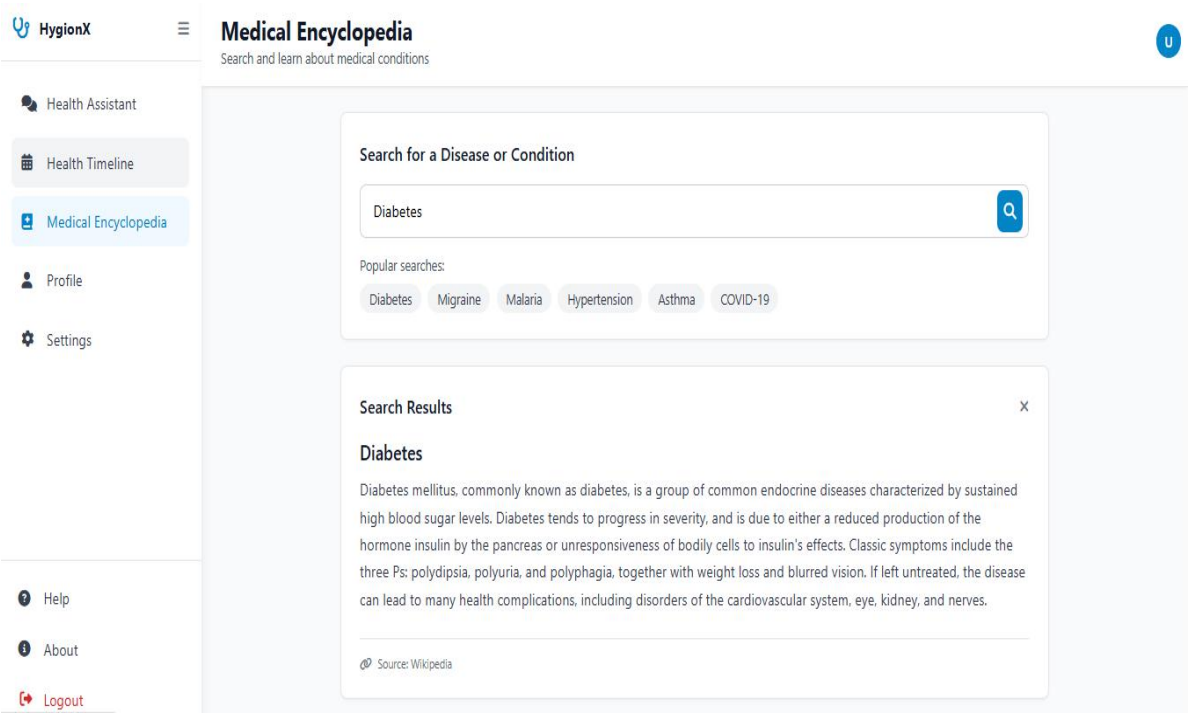

Fig 9.1.5 : Medical Encyclopedia Interface

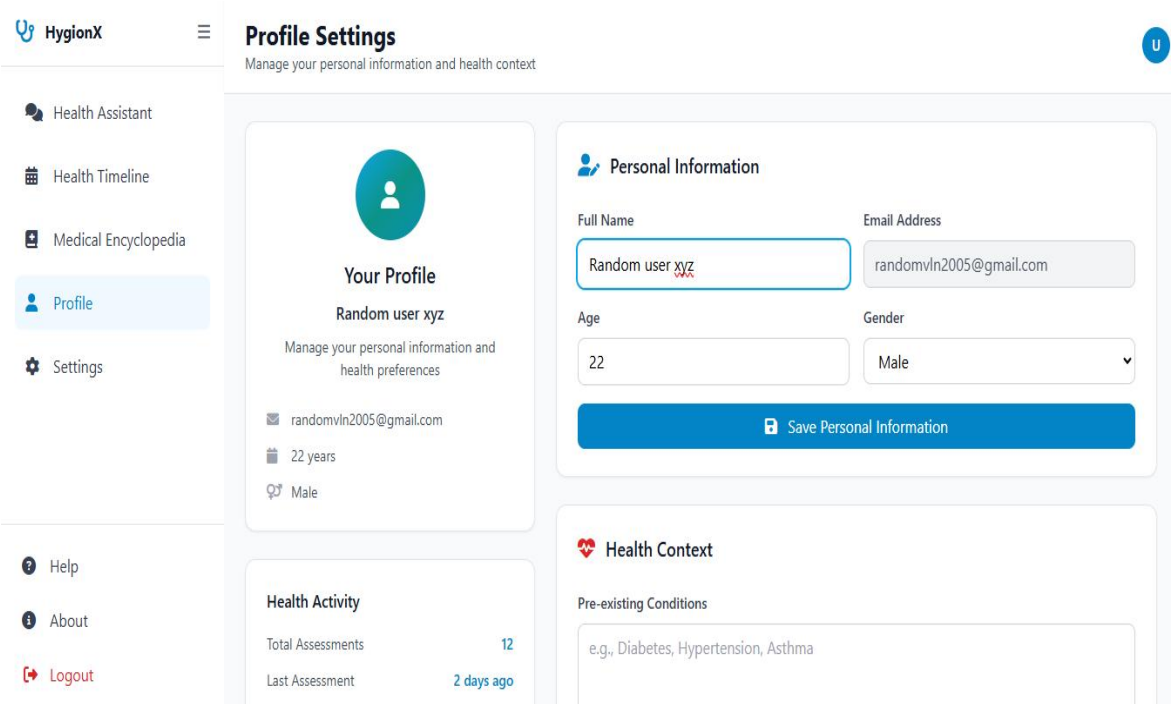

Fig 9.1.6 : Profile Interface

9.2 RESULTS

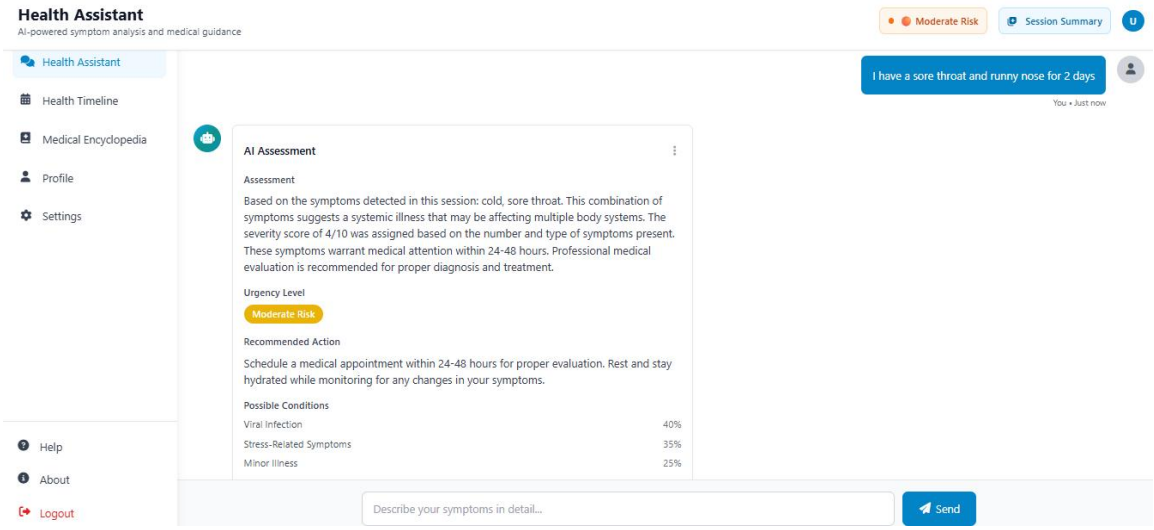

Fig 9.2.1 : Chat Output Interface

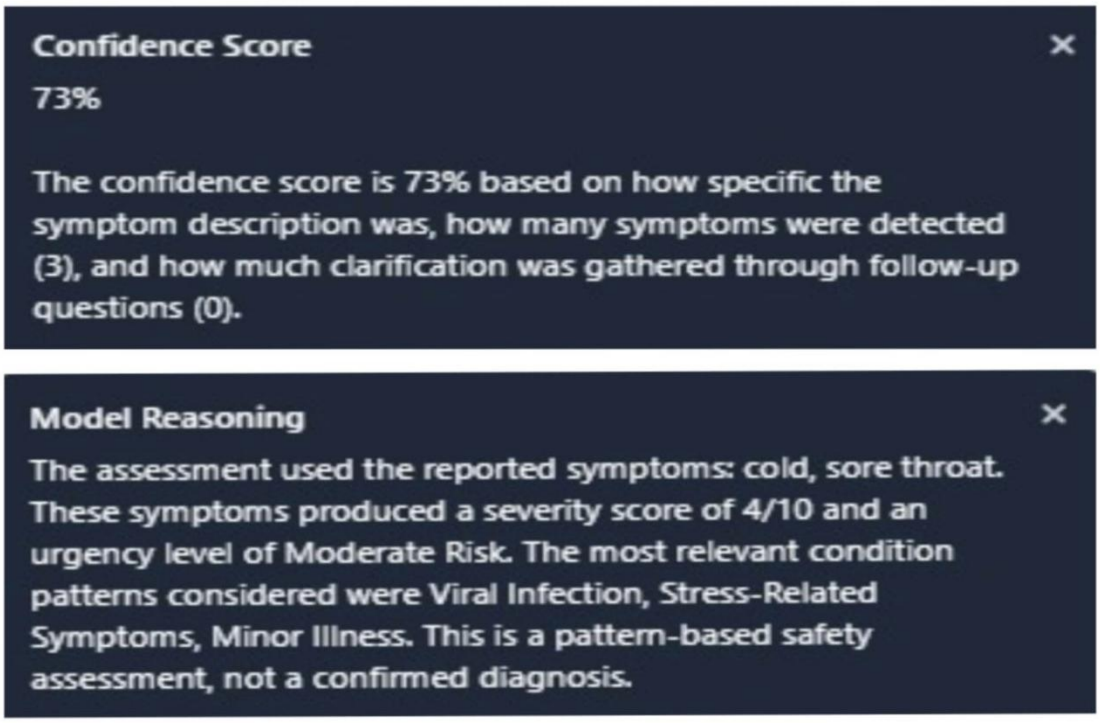

Fig 9.2.2 : Confidence scores and reasoning Interface

Chapter 10
FUTURE ENHANCEMENTS

The HygionX system provides a strong foundation for an AI-based medical triage assistant; however, several enhancements can be incorporated to further improve its capabilities and real-world applicability.

One of the major future enhancements is the integration of advanced domain-specific language models such as BioBERT, ClinicalBERT, or other medical LLMs. These models can improve the system's ability to understand complex medical queries, interpret user inputs more accurately, and generate more context-aware responses.

Another important enhancement is the incorporation of real-time medical knowledge bases and APIs. By connecting the system to trusted medical sources, the chatbot can provide updated medical information and more accurate recommendations aligned with current healthcare standards.

The system can also be enhanced by supporting multilingual interaction, making it accessible to a wider population and improving usability for users from different linguistic backgrounds.

Personalization can be further improved by incorporating deeper user health profiling, including past medical history, allergies, and previous interactions, enabling more precise and individualized triage results.

Integration with wearable devices and health monitoring systems can further enhance the system by enabling real-time data analysis and early detection of potential health issues.

Additionally, security and privacy can be strengthened by implementing advanced encryption and secure authentication mechanisms to protect sensitive health data.

The system can also be extended to support voice-based interaction, improving accessibility for elderly users and individuals with disabilities.

Finally, integrating telemedicine features such as doctor consultation and appointment booking can transform HygionX into a more comprehensive digital healthcare solution.

Chapter 11
CONCLUSION

The HygionX system successfully demonstrates the application of artificial intelligence in delivering preliminary medical guidance through an interactive, user-friendly chatbot interface. The system is designed to interpret user-reported symptoms, conduct a structured triage process, and generate meaningful health assessments along with appropriate recommendations, thereby improving accessibility to basic healthcare support. By integrating natural language processing, rule-based logic, and optional machine learning components, HygionX effectively understands user inputs, extracts relevant symptoms, and classifies the level of risk into categories such as low, moderate, high, or emergency. The system maintains conversational context across multiple interactions, enabling more accurate and consistent analysis through follow-up questioning and symptom aggregation.

A key strength of the system lies in its emphasis on explainability and transparency. Along with the triage results, HygionX provides a clear explanation of the reasoning behind its predictions. Additionally, the inclusion of a confidence score for each assessment plays a crucial role in conveying the reliability of the output, allowing users to make informed decisions, especially when professional consultation is recommended.

Furthermore, the system is designed with flexibility and robustness, supporting both ML-assisted and fallback modes to ensure uninterrupted functionality even in resource-constrained environments. Security features such as authentication and controlled data handling further ensure the safe management of sensitive user information.

In conclusion, HygionX serves as a reliable AI-based medical triage assistant that bridges the gap between basic symptom checking and intelligent healthcare support. While it does not replace professional medical diagnosis, it provides valuable preliminary insights, promotes informed decision-making, and contributes to enhancing digital healthcare accessibility and user awareness.

Chapter 12
BIBILOGRAPHY

1. Jegadeesan, R., Srinivas, D., Umapathi, N., Karthick, G., & Venkateswaran, N. (2023). Personal healthcare chatbot for medical suggestions using artificial intelligence and machine learning. *European Chemical Bulletin*, 12(S3).
2. Bickmore, T. W., & Giorgino, T. (2006). Health dialog systems for patients and consumers. *Journal of Biomedical Informatics*, 39(5), 556–571.
3. Laranjo, L., Dunn, A. G., Tong, H. L., et al. (2018). Conversational agents in healthcare: A systematic review. *Journal of the American Medical Informatics Association*, 25(9), 1248–1258.
4. Rajkomar, A., Dean, J., & Kohane, I. (2019). Machine learning in medicine. *New England Journal of Medicine*, 380(14), 1347–1358.
5. Topol, E. (2019). High-performance medicine: The convergence of human and artificial intelligence. *Nature Medicine*, 25(1), 44–56.
6. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *NAACL-HLT*.
7. Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*.
8. Miner, A. S., Milstein, A., & Hancock, J. T. (2017). Talking to machines about personal mental health problems. *JAMA*, 318(13), 1217–1218.
9. WHO (World Health Organization). (2020). Digital health interventions for health system strengthening.
10. FDA (Food and Drug Administration). (2021). Artificial Intelligence and Machine Learning in Software as a Medical Device.
11. Chen, M., & Decary, M. (2020). Artificial intelligence in healthcare: An essential guide for health leaders. *Healthcare Management Forum*, 33(1), 10–18.

12. Bibault, J. E., Chaix, B., Guillemassé, A., et al. (2019). A chatbot versus physicians to provide information for patients with breast cancer. *Journal of Medical Internet Research*, 21(11).
13. Denecke, K., et al. (2019). Artificial intelligence for patient support: A review of AI chatbots in healthcare. *Health Informatics Journal*.
14. Shortliffe, E. H., & Sepúlveda, M. J. (2018). Clinical decision support in the era of artificial intelligence. *JAMA*, 320(21), 2199–2200.
15. Jiang, F., Jiang, Y., Zhi, H., et al. (2017). Artificial intelligence in healthcare: Past, present and future. *Stroke and Vascular Neurology*, 2(4), 230–243.
16. Beam, A. L., & Kohane, I. S. (2018). Big data and machine learning in healthcare. *JAMA*, 319(13), 1317–1318.
17. Obermeyer, Z., & Emanuel, E. J. (2016). Predicting the future — Big data, machine learning, and clinical medicine. *New England Journal of Medicine*, 375(13), 1216–1219.
18. Razzaki, S., Baker, A., Perov, Y., et al. (2018). A comparative study of artificial intelligence and human doctors for triage and diagnosis. *npj Digital Medicine*, 1(1), 1–8.
19. Amisha, Malik, P., Pathania, M., & Rathaur, V. K. (2019). Overview of artificial intelligence in medicine. *Journal of Family Medicine and Primary Care*, 8(7), 2328–2331.
20. Davenport, T., & Kalakota, R. (2019). The potential for artificial intelligence in healthcare. *Future Healthcare Journal*, 6(2), 94–98.